JOHANNES KEPLER
UNIVERSITY LINZ

Author
Akintunde Osa Ojutiku

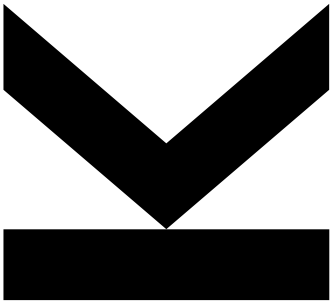
Submission
Institute of Machine Learning

Thesis Supervisor
Prof. Dr. Günter Klambauer

Assistant Thesis Supervisor
Niklas Schmidinger MSc.

Month Year
March, 2026

xLSTM For Bi-directional Protein Modelling Under Resource Constraints



Master's Thesis
to confer the academic degree of

Master of Science
in the Master's Program

Artificial Intelligence

JOHANNES KEPLER
UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at
DVR 0093696

Abstract

Protein language models learn sequence representations through self-supervised training on large unlabelled datasets. Although recent Transformer-based models have achieved impressive performance, their training demands substantial computational resources, limiting accessibility for many research groups. This raises a fundamental question: which architectures and design choices make the most effective use of limited training time and resources?

This thesis objectives addresses this question through the lens of *cramming*, a paradigm that evaluates model architectures by their learning efficiency under severe resource constraints rather than their asymptotic performance [10, 11]. Specifically, we compare Transformer-based (ESM2) and recurrent (mLSTM) architectures for protein language modelling when trained from scratch on a single GPU for 24 hours. All models are trained on UniRef50 sequences using masked language modelling, with architectural variants matched to approximately 8 million parameters.

To ensure fair comparison, all experiments use identical optimisation schedules, data pipelines, and hardware constraints. We evaluate both upstream performance through validation perplexity and downstream transfer learning on protein property prediction tasks from the FLIP benchmark. For the mLSTM architecture, we systematically explore depth-width trade-offs to understand how recurrent models scale under fixed parameter budgets.

Our results demonstrate that both architectures train stably under cramming constraints. Deeper mLSTM configurations consistently outperform shallow variants when parameters are held constant, indicating that recurrent models benefit from increased depth even with limited training. However, the Transformer baseline achieves lower validation perplexity (12.81 vs. 13.29) and stronger downstream performance (0.58 vs. 0.50 average Spearman correlation), with the gap particularly pronounced on tasks requiring long-range dependency modelling.

These findings extend efficiency-oriented analysis beyond Transformer-centric approaches and provide practical guidance for developing protein language models in resource-limited settings. By explicitly defining performance in terms of learning efficiency rather than compute-unlimited optimisation, this work contributes to more accessible and sustainable practices in computational protein modelling.

Contents

Abstract **i**

1 Introduction **1**

1.1 Background and Motivation 1
1.2 Problem Statement 2
1.3 Research Aim and Objectives 3
1.4 Research Questions 3
1.5 Research Design and Methodology Overview 4
1.6 Conceptual and Experimental Framework 5
1.7 Assumptions, Constraints, and Limitations 7
1.8 Contributions and Significance 7
1.9 Summary 7

2 Literature Review **9**

2.1 Conceptual Framework 9
2.2 Protein Language Models: Foundations and Evolution 11
2.2.1 From handcrafted features to learned representations 12
2.2.2 Self-supervised learning and masked language modelling 12
2.2.3 Data scale and standard pre-training corpora 13
2.2.4 Recurrent models to Transformers 13
2.3 Transformer-Based Protein Language Models 14
2.3.1 Transformer formulation for protein masked language modelling 15
2.3.2 Computational cost and scaling 16
2.3.3 Efficiency-aware perspective and motivation for alternatives . . 17
2.4 Efficient Training and the Cramming Paradigm 18
2.5 Recurrent Neural Networks in Protein Language Modelling 20
2.6 Modern Recurrent Architectures and the xLSTM Family 22
2.6.1 Classical LSTM: Vector-valued memory 22
2.6.2 Multiplicative LSTM: Matrix-valued memory and content-based retrieval 23
2.6.3 Chunkwise recurrence and computational structure 25
2.6.4 Compatibility with masked language modelling 26
2.6.5 Positioning within efficiency-oriented modelling 26
2.7 Efficient Sequence Modelling Beyond Attention 27
2.8 Depth–Width Trade-Offs Under Fixed Budgets 27

2.9 Summary and Research Gap 29

3 Methodology **31**

3.1 Methodological Approach and Experimental Design 32
3.1.1 Research Design Overview 32
3.1.2 Research Objectives 33
3.1.3 Experimental Constraints and Justification 34
3.2 Dataset and Data Acquisition 35
3.2.1 Upstream Pre-training Dataset: UniRef50 35
3.2.2 UniRef50 Access and Preprocessing Pipeline 35
3.2.3 Sequence Length Filtering 36
3.2.4 Training and Validation Split 36
3.2.5 Downstream Evaluation Dataset: FLIP Benchmark 36
3.2.6 FLIP Tasks Used in This Study 36
3.2.7 Downstream Data Usage Protocol 37
3.3 Tokenization and Input Representation 37
3.4 Upstream Training Objective: Masked Language Modelling 37
3.5 Masking Strategy 38
3.6 Upstream Loss Function 38
3.7 Downstream Evaluation Tasks 38
3.8 Downstream Prediction Objective 39
3.9 Experimental Design 39
3.9.1 Phase 1: Baseline Architectural Comparison 40
3.9.2 Phase 2: Depth Scaling Analysis 40
3.9.3 Hyperparameter Exploration 40
3.9.4 Complete Experimental Matrix 41
3.10 Model Architectures 42
3.10.1 ESM2-Style Transformer Baseline 42
3.10.2 mLSTM Architecture Variants 43
3.10.3 Bidirectional Processing Adaptation 43
3.10.4 Depth Variant Specifications 43
3.11 Training Paradigm: Cramming-Style Optimisation 44
3.11.1 Cramming-Style Training 44
3.11.2 Implications for Model Training and Evaluation 45
3.11.3 Relation to the Training Algorithm 45
3.12 Training Configuration 46
3.12.1 Hardware Environment and Computational Constraints 46
3.12.2 Training Duration and Budget 46
3.12.3 Batching and Gradient Accumulation 46

3.12.4 Optimisation Settings . . . 46
3.12.5 Learning Rate Schedule . . . 47
3.12.6 Training Stability Mechanisms . . . 47
3.12.7 Checkpoint and Logging Configuration . . . 48
3.13 Training Procedure and Algorithm . . . 48
3.13.1 Training Algorithm . . . 49
3.13.2 Implementation Details . . . 49
3.14 Evaluation Metrics and Performance Assessment . . . 49
3.14.1 Training and Validation Loss . . . 49
3.14.2 Validation Perplexity . . . 51
3.14.3 Training Efficiency Indicators . . . 52
3.14.4 Training Stability Indicators . . . 52
3.15 Downstream Evaluation Protocol . . . 52
3.15.1 Motivation and Evaluation Scope . . . 53
3.15.2 Frozen Representation Extraction . . . 53
3.15.3 Downstream Prediction Heads . . . 53
3.15.4 Downstream Tasks and Benchmarks . . . 54
3.16 Reproducibility and Transparency . . . 54
3.17 Ethical Considerations . . . 55
3.18 Summary . . . 55

4 Results and Key Findings **56**
4.1 Dataset Context and Protein Sequence Constraints . . . 56
4.2 Training Monitoring and Execution Evidence . . . 57
4.2.1 Logging and Experiment Tracking . . . 57
4.2.2 Learning Rate Schedule Behaviour . . . 57
4.3 Validation Frequency and Monitoring . . . 57
4.3.1 Checkpointing and Run Completion . . . 58
4.3.2 Training Stability Mechanisms and Observed Stability . . . 58
4.4 Validation Loss and Perplexity Results . . . 58
4.5 Model Comparison Results: ESM2 vs. mLSTM . . . 60
4.6 Depth Exploration Results: mLSTM . . . 61
4.6.1 Hyperparameter Sensitivity Analysis for mLSTM . . . 63
4.7 Supplementary Upstream Results . . . 64
4.7.1 Complete Checkpoint Performance . . . 64
4.7.2 Final Upstream Performance Results . . . 65
4.7.3 Comparative Analysis . . . 67
4.7.4 Summary of the Upstream Findings . . . 68
4.8 Downstream Evaluation . . . 69

4.8.1 Downstream Tasks and Evaluation Protocol 69
4.8.2 Quantitative Results 69
4.8.3 Architectural Comparison Across Tasks 70
4.8.4 Task-Specific Analysis 70
4.8.5 Relationship Between Upstream and Downstream Performance . 71
4.8.6 Architectural Implications 71
4.8.7 Summary 73

5 Discussion and Conclusion **74**
5.1 Overview of Findings 74
5.2 Training Stability and Protein-Specific Considerations 74
5.3 Architectural Comparison Under Cramming Constraints 75
5.4 Depth–Width Trade-Offs in mLSTM Models 75
5.5 Downstream Transfer and Functional Representation Quality 76
5.6 Relationship Between Upstream and Downstream Performance 76
5.7 Implications for Efficient Protein Language Modelling 77
5.8 Limitations 77
5.9 Future Work 77
5.10 Key Contributions of the Thesis 78
5.11 Conclusion 78

References **80**

List of Figures

1.1 Conceptual framework of the experimental study, linking protein sequence data[2, 32] processed via masked language modelling[8] through cramming-style constraints[10, 11] to mLSTM[3] and Transformer[19, 34] architectures, optimisation dynamics[17], and upstream and downstream evaluation outcomes[6]. 6

4.1 Training loss (left) and validation perplexity (right) across optimisation steps for the parameter-matched ESM2 (6-layer baseline) and mLSTM 6-layer, trained on UniRef50 protein sequences under the MLM objective. 59
4.2 Validation perplexity comparison between ESM2-8M (6-layer baseline) and mLSTM-8M (6-layer) on UniRef50 protein sequences under the masked language modelling objective. The dashed line indicates the reported perplexity of the pre-trained ESM2-8M baseline. 60
4.3 Validation perplexity trends for mLSTM depth exploration across learning rates and warm-up settings. 62
4.4 Training loss (left) and validation perplexity (right) for the 6-layer mLSTM under different learning rate and warm-up configurations. 63
4.5 Spearman rank correlation comparison between ESM2 and mLSTM across downstream fitness prediction tasks. 72
4.6 Predicted versus true fitness for downstream tasks. Stronger alignment with the diagonal corresponds to higher rank correlation. 72

List of Tables

2.1 Comparison of classical LSTM and mLSTM formulations. 26

3.1 Complete experimental matrix showing all training configurations. Each cell represents a single training run. 41
3.2 mLSTM architectural configurations for depth scaling analysis. Hidden dimensions are adjusted to maintain approximately 8M parameters across all depth variants. 43

4.1 Final upstream validation results after 10,000 updates on UniRef50 protein MLM. Lower is better for both loss and perplexity. 61
4.2 Validation perplexity at logged checkpoints for all training configurations. Lower values indicate better upstream masked language modelling performance. 65
4.3 Final upstream validation performance after 10,000 optimisation steps. Lower is better for both validation perplexity and validation loss. All models are approximately parameter-matched (∼8M parameters). . . . 66
4.4 Statistical summary of validation perplexity across all configurations at each checkpoint. Lower values indicate better performance. 66
4.5 Performance comparison between ESM2 baseline and mLSTM variants at step 10,000. Gap represents the perplexity difference from ESM2 baseline. 67
4.6 Validation perplexity progression for ESM2 and best/worst mLSTM configurations across training checkpoints. 67
4.7 Downstream protein fitness prediction performance using frozen representations and linear regression heads. Higher Spearman correlation and lower MSE indicate better performance. 70

1 Introduction

1.1 Background and Motivation

Recent advances in high-throughput sequencing and large-scale metagenomic sampling have resulted in an unprecedented growth in available protein sequence data[21, 30]. This rapid expansion has fundamentally transformed computational biology, shifting emphasis away from handcrafted features toward data-driven representation learning[36]. Within this paradigm, protein language models (pLMs) have emerged as a powerful approach for learning meaningful representations directly from raw amino acid sequences by treating proteins as a form of biological language shaped by evolutionary and biophysical constraints[19, 28].

Protein language models are typically trained on large, unlabelled sequence databases such as UniRef50 and UniProt[2, 32] using self-supervised objectives[8]. Through masked language modelling (MLM)[8], these models learn contextual embeddings that implicitly capture structural motifs, evolutionary conservation, and functional relationships without requiring explicit annotations[19, 28]. Such representations have been shown to support a wide range of applications, including structure prediction[19, 21], mutational effect estimation[22], and functional annotation[28].

Most recent progress in protein language modelling has been driven by Transformer-based architectures[9, 19]. Models such as ESM2 rely on multi-head self-attention[34] to capture long-range dependencies across protein sequences and achieve strong performance primarily through large-scale parameterisation[19]. However, this success has come at a substantial computational cost. Training state-of-the-art Transformer-based protein models typically requires large GPU clusters and extended training times, limiting their accessibility to well-resourced research environments and restricting systematic architectural exploration[9, 19].

To address these limitations, the cramming paradigm was introduced as an efficiency-oriented training framework. Originally developed in natural language processing, cramming demonstrates that competitive language models can be trained within strict computational budgets, commonly under 24 hours on a single GPU through careful architectural design, optimisation schedules, and data pipeline engineering[10, 11]. More recently, this paradigm has been extended to protein language modelling, where compact Transformer models trained under fixed time and hardware constraints were shown to approach the performance of substantially larger pre-trained models[10].

Despite these advances, existing cramming-based studies have focused almost exclusively on Transformer architectures. In contrast, modern recurrent architectures re-

main largely underexplored in protein language modelling, despite offering linear-time sequence processing and favourable memory characteristics[3, 12]. Recent developments in the Extended Long Short Term Memory (xLSTM) family of recurrent models have demonstrated that gated recurrent mechanisms can achieve competitive language modelling performance while maintaining linear computational complexity[3]. Within this family, the multiplicative Long Short Term Memory (mLSTM) combines exponential gating with matrix-based memory, enabling efficient sequence modelling with strong representational capacity[3].

These properties make mLSTM a promising candidate for protein language modelling under strict computational constraints. However, unlike Transformer-based models[19, 34], principled architectural scaling guidelines for modern recurrent architectures remain poorly understood[3, 12], particularly in the context of protein sequences. This gap motivates a systematic investigation of how architectural depth influences the performance and efficiency of mLSTM-based protein language models under the cramming paradigm[10, 11].

1.2 Problem Statement

The effectiveness of cramming-style training depends critically on architectural design choices[10, 11]. When training time, memory, and parameter count are tightly constrained, suboptimal architectural configurations cannot be compensated for through longer training or extensive hyperparameter tuning[11]. Under such conditions, the allocation of model capacity between architectural depth and width becomes especially consequential[16].

This trade-off manifests differently across architectural families. In Transformer-based models, increasing depth enables hierarchical refinement of attention patterns[16, 34], although excessive depth may introduce optimisation difficulties or diminishing returns[15, 16]. In recurrent architectures such as mLSTM, depth affects not only layer-wise transformations but also the propagation of information across time steps[3, 14]. While mLSTM incorporates gating mechanisms designed to mitigate vanishing gradients, increasing depth under a fixed parameter budget necessarily reduces layer width, potentially constraining representational capacity[3, 13].

Although scaling behaviour in Transformer-based language models has been extensively studied[15, 16], these findings do not directly transfer to recurrent architectures due to fundamental differences in computation, inductive bias, and optimisation dynamics[3, 12]. For mLSTM in particular, which employs chunk-wise recurrent operations with linear complexity [3], the interaction between depth, width, and training behaviour under cramming constraints remains poorly understood.

This gap is especially pronounced in protein language modelling, where sequence

lengths, vocabulary structure, and data distributions differ substantially from those of natural language [19, 28]. While prior work has demonstrated that Transformer-based protein language models can be effectively trained under cramming constraints [10, 11], no comparable study has systematically examined architectural depth scaling in modern recurrent protein language models. Consequently, researchers currently lack empirical guidance for designing efficient recurrent (xLSTM) models for protein language modelling under fixed computational budgets[3].

1.3 Research Aim and Objectives

The purpose of this study is to systematically investigate depth–width trade-offs in mLSTM-based protein language models trained under the cramming paradigm[10, 11]. The primary focus is on understanding how architectural depth changes while maintaining a fixed parameter budget of approximately eight million parameters affects training stability, convergence behaviour, and upstream masked language modelling performance[8].

To contextualise the performance of mLSTM models, a fixed Transformer-based protein language model following the ESM2 configuration is included as a reference baseline[19]. This baseline is not subjected to architectural variation and serves solely as a point of comparison, as Transformer scaling behaviour under cramming constraints has been characterised in prior work[10, 11].

In addition to upstream evaluation, this study incorporates a downstream analysis to assess the transferability of representations learned under cramming-style constraints. Downstream evaluation is conducted using protein property prediction tasks from the FLIP benchmark[6] and serves to examine whether architectural differences observed during upstream training translate into differences in practical task performance.

All mLSTM variants are trained under identical upstream conditions, including matched parameter counts, consistent optimisation schedules[17], and strict single-GPU training budgets[11]. Warm-up duration is varied in a controlled manner to examine its interaction with architectural depth. Together, the upstream and downstream analyses provide a comprehensive assessment of architectural efficiency under extreme resource constraints.

1.4 Research Questions

This thesis investigates the effectiveness of mLSTM-based architectures[3] for protein language modelling under cramming-style training constraints[10, 11], using a well-established Transformer-based protein language model as a fixed reference baseline[19].

The study examines both upstream masked language modelling behaviour[8] and downstream transfer performance under limited computational resources.

The research is guided by the following Research Questions (RQ):

- RQ1: To what extent can an mLSTM-based protein language model achieve competitive upstream masked language modelling performance compared to an ESM-style Transformer when both models are trained under identical cramming-style constraints and matched parameter budgets?
- RQ2: How does architectural depth; considering shallow (1-layer), baseline (6-layer), and deep (12-layer) configurations, affect the upstream masked language modelling performance of mLSTM models when the total parameter count and training budget are held constant?
- RQ3: How do training efficiency, convergence behaviour, and stability of mLSTM models compare to those of a fixed ESM-style Transformer baseline under cramming-style training?
- RQ4: To what extent do representations learned under cramming-style constraints transfer to downstream protein property prediction tasks from the FLIP benchmark, and how do mLSTM and Transformer-based models compare in this setting?

These research questions are addressed using training and validation loss curves, final validation perplexity values, and observed convergence behaviour. The emphasis is on identifying empirical patterns.

1.5 Research Design and Methodology Overview

This research adopts a quantitative, experimental design based on controlled architectural variation. The study systematically varies the depth and width of mLSTM-based[3] protein language models while holding all other factors constant, including total parameter count, dataset, training duration, and optimisation strategy[10, 11]. This controlled setup enables isolation of the effects of architectural depth within the cramming regime[11].

All models are trained from scratch using a masked language modelling objective[8] on the UniRef50 dataset[2, 32]. Training is conducted under strict single-GPU constraints using fixed batch sizes and gradient accumulation to ensure hardware-level comparability[10, 11]. Identical optimiser configurations[17] and learning rate schedules are applied across all models, with the exception of warm-up duration, which is varied in a controlled manner.

Parameter counts are matched across depth variants by adjusting the hidden dimension, such that shallower models employ wider layers while deeper models use narrower layers[3, 16]. A fixed six-layer Transformer model based on the ESM2 configuration[19] is included as a reference baseline and is trained under identical upstream conditions.

1.6 Conceptual and Experimental Framework

The conceptual framework of this study links architectural depth to training behaviour under constrained optimisation conditions. In recurrent architectures such as mLSTM[3], depth influences both hierarchical feature transformation across layers and the propagation of information through sequence positions[3, 14]. Under cramming-style training, where computational budgets are fixed, these effects become particularly pronounced[10, 11].

Increasing depth may enhance representational capacity through hierarchical composition but also introduces longer optimisation paths that can hinder convergence under limited training schedules[11, 16]. Conversely, shallow but wider configurations allocate more capacity within individual layers, potentially improving early convergence and training stability at the cost of reduced hierarchical abstraction[16, 18].

Depth also interacts with computational efficiency. Under fixed wall-clock budgets, deeper models incur higher per-step computational cost, reducing the total number of effective updates achievable within the training window[10, 11]. Consequently, architectural depth, width, optimisation efficiency, and training stability interact in complex ways under cramming constraints[10, 11].

The inclusion of a fixed Transformer baseline provides a comparative reference that highlights differences between recurrent and attention-based inductive biases under identical constraints, enabling architectural effects to be interpreted independently of training configuration[19, 34].

Figure 1.1 illustrates the conceptual framework guiding this study, showing how protein sequences from UniRef50[2, 32] are modeled under cramming-style constraints[10, 11] using a masked language modelling objective[8] with depth-scaled mLSTM variants[3] and a fixed Transformer baseline[19].

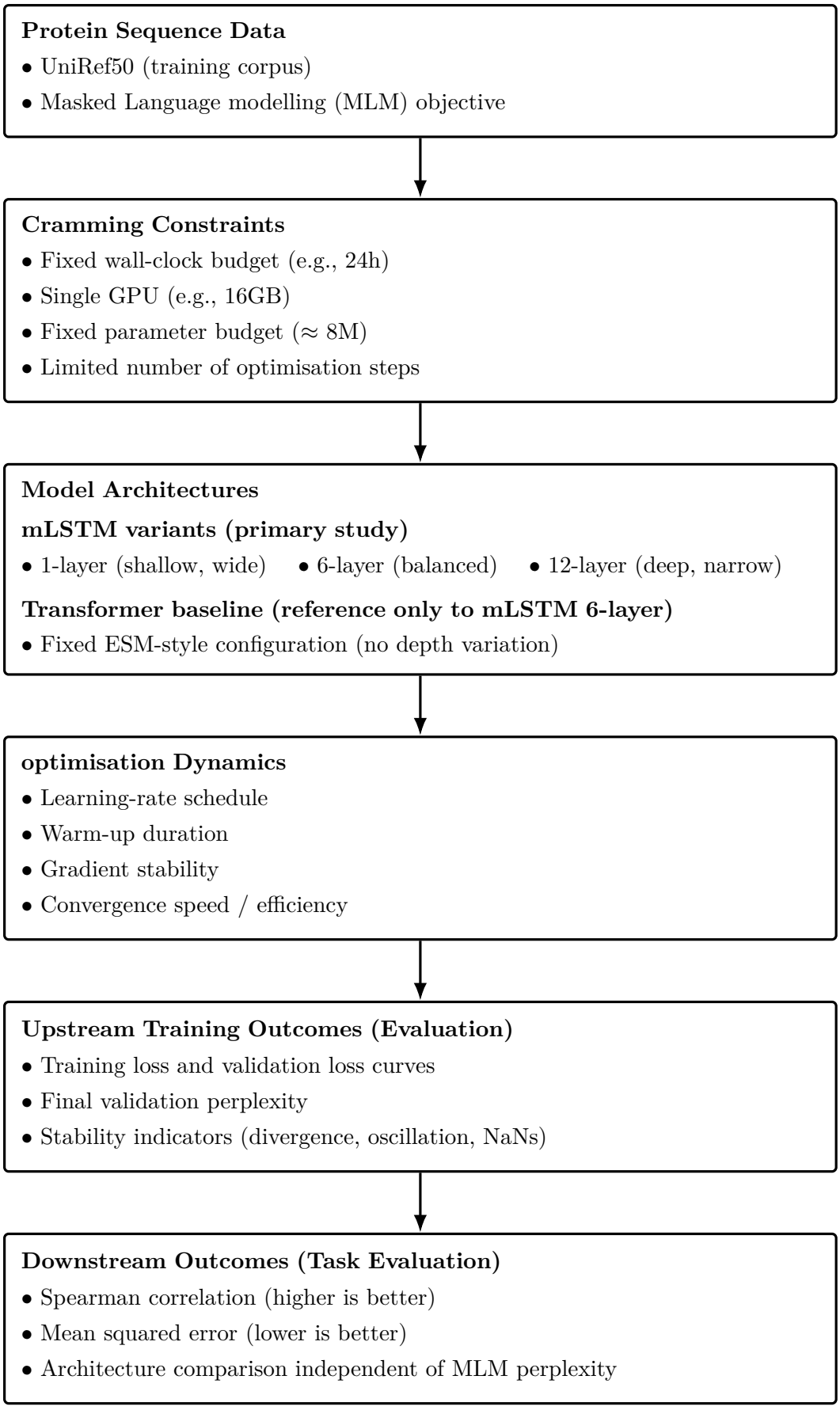


Figure 1.1: Conceptual framework of the experimental study, linking protein sequence data[2, 32] processed via masked language modelling[8] through cramming-style constraints[10, 11] to mLSTM[3] and Transformer[19, 34] architectures, optimisation dynamics[17], and upstream and downstream evaluation outcomes[6].

1.7 Assumptions, Constraints, and Limitations

This study operates under several assumptions. First, masked language modelling loss and validation perplexity are assumed to be meaningful indicators of upstream representation quality in protein language models[19, 28]. While downstream evaluation can provide additional insights, upstream metrics are considered sufficient for architectural comparison under constrained training regimes[10, 11].

Second, the use of standardised optimisation schedules across configurations is assumed to enable fair comparison[11]. Although depth-specific hyperparameter tuning might improve individual models, such tuning would obscure the architectural effects of interest[11, 16]. Parameter matching across configurations is also assumed to reasonably control for model capacity[15, 16].

Several limitations must be acknowledged. Experiments are conducted on a single dataset with fixed sequence length constraints, and each configuration is trained only once due to computational limitations[10, 11]. These design choices reflect the principles of the cramming paradigm and define clear boundaries for interpreting the results[10, 11].

1.8 Contributions and Significance

This study contributes to protein language modelling research across scientific, methodological, and practical dimensions. Scientifically, it provides empirical insight into depth–width trade-offs in modern recurrent protein language models[3], an area that remains under-explored relative to Transformer-based approaches[9, 19].

Methodologically, the work extends the cramming paradigm[10, 11] beyond Transformers, demonstrating that recurrent architectures[3] can be systematically analyzed under extreme resource constraints. The experimental design offers a reproducible framework for architectural evaluation in efficiency-oriented settings[11].

Practically, the findings provide guidance for researchers designing protein language models under limited computational budgets[10, 11], supporting informed architectural choices without reliance on large-scale infrastructure[9, 19].

1.9 Summary

This chapter introduced the motivation, scope, and design of the study. The research problem was framed around unresolved depth–width trade-offs in mLSTM-based[3] protein language models trained under cramming constraints[10, 11]. The chapter articulated clear research questions, described the experimental nature of the study, and presented a conceptual framework linking architectural depth to training dynamics.

Key assumptions and limitations were explicitly stated to define the boundaries of interpretation, and the significance of the work was discussed across multiple dimensions. By focusing on mLSTM[3] depth scaling while using a fixed Transformer baseline[19, 34] for reference, the study establishes a clear scope appropriate for the experiments under limited computational resources[10, 11]. The following chapter situates this work within the broader literature on protein language models[9, 19, 28], architectural scaling[15, 16], efficient training methodologies[10, 11], and evaluations[6].

2 Literature Review

2.1 Conceptual Framework

This study is grounded in a conceptual framework that explains how architectural design choices interact with optimisation dynamics to determine training outcomes in protein language models trained under strict computational constraints. Rather than viewing model performance as an asymptotic property achieved through large-scale pre-training, this framework emphasises how architectural decisions shape what can be learned within a fixed and limited training horizon[10, 11].

Central to this framework is the cramming paradigm, which defines a training regime in which computational resources are explicitly bounded. Under cramming-style training, models are trained from scratch within a fixed wall-clock budget, typically 24 hours using a single GPU with limited memory[10, 11]. As a result, the number of optimisation steps achievable during training is strictly limited. Prior work has demonstrated that, under such constraints, model performance depends not only on representational capacity but also on optimisation efficiency, numerical stability, and early convergence behaviour[10, 11]. Architectural inefficiencies cannot be compensated for through extended training or extensive hyperparameter search, fundamentally altering the effective optimisation landscape relative to large-scale pre-training.

Within this constrained regime, neural architectures are conceptualized as the combination of an inductive bias and a computational mechanism. The inductive bias determines the types of dependencies a model is predisposed to capture, while the computational mechanism governs how efficiently these dependencies can be learned on available hardware. Transformer-based architectures, such as ESM-style models, encode a bias toward global context modelling through multi-head self-attention, enabling direct interactions between all token pairs in a sequence[19, 34]. In contrast, recurrent architectures encode a sequential inductive bias, in which contextual information is accumulated through gated state updates. Modern recurrent variants, including the xLSTM family, extend this paradigm through improved gating mechanisms and matrix-based memory structures that enhance expressivity while preserving linear-time recurrence[3, 12].

Under cramming constraints, the computational profile of an architecture becomes especially salient. Although recurrent models exhibit linear-time complexity with respect to sequence length, their practical efficiency depends on kernel implementations, parallelisation strategies, and memory access patterns[3, 7]. Conversely, Transformer models, despite their quadratic theoretical complexity, benefit from highly optimised

attention kernels that can yield high computational efficiency on modern GPUs for moderate sequence lengths[7, 33]. Consequently, architectural effectiveness under constrained training cannot be inferred from asymptotic complexity alone but must instead be evaluated in terms of effective learning progress achieved within a fixed time window[10, 26].

A central construct in this framework is the depth–width trade-off under a fixed parameter budget. When total parameter count is constrained, increasing architectural depth necessarily requires reducing layer width, and vice versa. Depth enables hierarchical feature composition across layers, which can enhance expressivity when sufficient optimisation budget is available[16]. Width, by contrast, determines per-layer representational capacity and directly influences gradient signal strength, often facilitating faster early convergence, improved numerical stability and properties that are particularly valuable in short training regimes[16, 18].

Under cramming-style training, this trade-off becomes especially consequential. Deeper models incur higher computational cost per update and often require more careful optimisation to remain stable, reducing the number of effective parameter updates achievable within the fixed training window[11, 15]. Shallow–wide models may converge more rapidly due to stronger gradient signals and shorter optimisation paths but may saturate earlier and fail to capture more complex hierarchical dependencies. The framework therefore treats architectural depth and width as competing allocations of limited capacity, whose optimal balance depends critically on the constrained optimisation regime.

Optimisation dynamics mediate the relationship between architectural design and training outcomes. Learning rate warm-up plays a particularly important role under cramming constraints, as unstable early updates can irreversibly degrade training progress[11, 18]. Prior work has shown that deeper architectures are especially sensitive to warm-up duration, as insufficient warm-up can lead to gradient instability and poor convergence[3, 18]. Although gating mechanisms in mLSTM mitigate some optimisation challenges, deep recurrent stacks remain sensitive to aggressive learning rate schedules when layer width is reduced to maintain a fixed parameter budget[3, 12].

The observable outcomes of interest in this study emerge from the interaction between architecture and optimisation under cramming constraints. These outcomes include training and validation loss trajectories, final validation perplexity, and qualitative indicators of training stability such as divergence or oscillatory behaviour. Validation perplexity is treated as the primary upstream performance metric, reflecting the model's ability to capture statistical regularities in protein sequences under a masked language modelling objective[8]. This choice aligns with established practice in protein language modelling, where upstream perplexity has been shown to correlate with the quality of learned representations[10, 19, 28].

Within the broader architectural landscape, Transformer-based protein language models represent the dominant paradigm, achieving strong performance through global attention mechanisms but at substantial computational cost[9, 19]. Modern recurrent architectures, including the xLSTM family and its mLSTM variant, represent a renewed effort to reconcile efficiency and expressivity through improved gating, matrix-based memory, and parallelisable recurrence[3]. In parallel, recent work on linear-time and sub-quadratic sequence models further highlights a broader shift toward architectures designed for efficient training under constrained resources[12, 26, 33].

By integrating these perspectives, the conceptual framework directly supports the research questions posed in Chapter 1. It provides a principled rationale for examining architectural depth in mLSTM-based protein language models under fixed computational budgets, explains why convergence speed and stability are critical evaluation dimensions, and clarifies the role of learning rate warm-up as an interacting factor rather than a peripheral tuning choice. The inclusion of a fixed Transformer baseline is justified as a reference point with well-characterized behaviour under cramming constraints[10, 19], enabling differences in training outcomes to be attributed to architectural inductive bias rather than training protocol.

In summary, this conceptual framework establishes a coherent theoretical foundation linking architectural depth, optimisation dynamics, and upstream training outcomes under cramming constraints. It positions depth–width trade-offs in mLSTM-based protein language models as a meaningful and underexplored research problem[3], grounded in recent advances in efficient sequence modelling[12, 26] and motivated by practical limitations in computational resources[10, 11]. This framework informs the subsequent literature review and provides the analytical lens through which empirical findings are interpreted in later chapters.

2.2 Protein Language Models: Foundations and Evolution

Protein language models (pLMs) emerged from the convergence of large-scale protein sequence databases, advances in neural sequence modelling, and self-supervised learning. Over the past two decades, improvements in efficient sequencing know-how and database curation have resulted in rapidly expanding collections of protein sequences, creating a systematic data rich environment where representation learning is particularly effective. This development shifted computational biology away from handcrafted features toward models that learn biologically informative representations directly from raw amino acid sequences[2, 27, 29, 36].

2.2.1 From handcrafted features to learned representations

Early computational approaches relied on handcrafted descriptors derived from physicochemical properties and evolutionary information, often computed from multiple sequence alignments and profiles[1, 27, 36]. While successful in targeted settings, these approaches required substantial domain expertise and tended to be tightly coupled to specific tasks, datasets, or protein families, limiting generalization and transfer. This motivated a move toward task-agnostic, learned representations that can be reused across problems without bespoke feature design[1, 36].

Motivated by analogies to natural language, proteins began to be treated as a biological "language" composed of sequence of elements (amino acids)[1, 28]. In this view, a protein sequence of length L is represented as

$$x = (x_1, x_2, \ldots, x_L), \quad x_i \in \mathcal{V},$$

where $\mathcal{V}$ is a finite vocabulary typically comprises of 20 canonical amino acids plus special elements. This formulation enables direct reuse of neural sequence modelling machinery developed for language and other discrete sequences[27, 28].

2.2.2 Self-supervised learning and masked language modelling

Self-supervised learning provides the key mechanism for training pLMs without experimental labels. Models are trained to solve prediction tasks derived from the sequences themselves. The most widely adopted objective is masked language modelling (MLM) popularly known in natural language processing (NLP) and later adapted to proteins, where the model reconstructs masked sequences from context. This objective is well aligned with protein sequence data because it encourages the model to learn contextual regularities without requiring annotation[8, 19, 28].

Formally, given a sequence x, sample a subset of masked positions

$$\mathcal{M} \subset \{1, \ldots, L\},$$

and construct the masked input sequence $\tilde{x}$ as

$$\tilde{x}_i = \begin{cases} [\text{MASK}], & i \in \mathcal{M}, \\ x_i, & \text{otherwise.} \end{cases}$$

A model with parameters θ predicts a conditional distribution over residues $p_\theta(x_i \mid \tilde{x})$. The MLM objective minimizes the negative log-likelihood on masked positions:

$$\mathcal{L}_{\text{MLM}}(\theta) = -\sum_{i \in \mathcal{M}} \log p_\theta(x_i \mid \tilde{x}).$$

Over a dataset $\mathcal{D}$, training corresponds to minimizing the expectation of this loss over sequences and masking patterns[8, 28].

During training, the model produces contextual hidden states (embeddings) at each position,

$$h_i = f_\theta(\tilde{x})_i,$$

which can be used as the sequence element representations, and can be aggregated for example, by mean pooling for sequence-level representations. A consistent empirical finding is that the representations learned by protein language models capture biologically meaningful structure. This is demonstrated by downstream analyses showing strong correlations with protein secondary structure, spatial residue contacts, and mutational effects, despite the models being trained using only sequence-level supervision[19, 22, 28].

2.2.3 Data scale and standard pre-training corpora

Large pLMs are typically pre-trained on broad sequence repositories such as UniProt and clustered subsets such as UniRef. UniRef50 is widely used because it reduces redundancy by clustering sequences at 50% identity while maintaining diversity, enabling efficient training on broad evolutionary variation without excessive duplication. This dataset choice has become standard in many protein language modelling pipelines and provides a common basis for comparing training strategies and architectures[2, 19, 32].

2.2.4 Recurrent models to Transformers

Early protein language models often used recurrent neural networks (RNNs), including LSTMs[14] and GRUs, reflecting their natural fit to sequential data. However, recurrent models can face challenges in capturing long-range dependencies and scaling efficiently to long proteins, motivating the shift toward attention-based architectures[1, 27, 28]. Transformers, enabled by self-attention[34], became dominant because they model long-range interactions directly and can be trained efficiently with modern parallel hardware.

Transformer-based protein language models, particularly the ESM family, demonstrated that scaling attention-based models and training on massive protein corpora can yield representations strongly associated with structure and function. However, achieving frontier performance typically requires substantial computational resources with large GPU clusters and long training times, which limits accessibility and motivates the efficiency-oriented framing introduced in earlier Chapter[9, 19, 28].

The growing computational cost of large pLMs has motivated methods that emphasises efficiency and constrained training. The cramming paradigm formalizes this setting by training under strict hardware and wall-clock budgets for example, training from scratch within 24 hours on a single GPU, making optimisation dynamics and ar-

chitectural efficiency decisive[10, 11]. This study adopts that perspective and focuses on what can be learned within a fixed training horizon rather than asymptotic scaling.

In parallel, renewed interest has emerged in architectures that may offer improved trade-offs under such constraints, including modern recurrent families and linear-time sequence models. Recent recurrent developments for example, the xLSTM aim to combine strong sequence modelling capacity with favourable computational and memory characteristics, making xLSTM plausible candidates for efficient protein MLM under the cramming regime. This motivates the thesis focus on recurrent depth scaling under fixed budgets, using a Transformer baseline only as a reference point[3, 10, 12].

2.3 Transformer-Based Protein Language Models

Transformer architectures have become the dominant paradigm in protein language modelling, largely due to their ability to capture long-range dependencies through self-attention mechanisms[8, 34]. Introduced originally for natural language processing, the Transformer replaces recurrent computation with parallel attention-based interactions, enabling each token in a sequence to attend directly to all others[34]. This property is particularly well suited to proteins, where residues distant in the primary sequence may interact closely in three-dimensional space and jointly determine structural stability and biological function[19, 28].

The application of Transformers to protein sequences was popularized by the Evolutionary Scale modelling (ESM) family of models. Early ESM models demonstrated that large Transformer architectures trained on massive protein databases can learn representations that implicitly encode secondary structure, contact maps, and evolutionary constraints without explicit supervision[27, 28]. Subsequent work culminated in ESM2, which introduced architectural refinements, improved training recipes, and substantially larger model scales. ESM2 models were trained on hundreds of millions of protein sequences and scaled to billions of parameters, achieving state-of-the-art performance across a wide range of protein prediction tasks, including structure prediction and mutational effect estimation[19, 21]. These results provide strong empirical evidence that self-attention is capable of modelling biologically meaningful long-range interactions and that scaling Transformer-based pLMs can yield systematic improvements in representation quality[8, 16, 19].

At the core of Transformer success is the self-attention mechanism, which enables direct modelling of pairwise interactions between all sequence positions[34]. Multi-head attention allows the model to learn multiple interaction patterns simultaneously, capturing diverse dependency structures within protein sequences[19, 34]. This capability is particularly valuable for proteins, where functional and structural relationships are often mediated by non-local interactions. As a result, attention-based models have

consistently outperformed earlier recurrent approaches on many downstream protein tasks[22, 28].

2.3.1 Transformer formulation for protein masked language modelling

To connect Transformer pLMs with the masked language modelling (MLM) objective introduced earlier, consider an input protein sequence

$$x = (x_1, \ldots, x_L), \tag{2.1}$$

where x_i are amino acid tokens and L is the sequence length. Let

$$H^{(0)} \in \mathbb{R}^{L \times d} \tag{2.2}$$

denote the initial token embeddings (optionally augmented with positional encodings), where d is the model dimension.

For Transformer layer ℓ, the input $H^{(\ell)} \in \mathbb{R}^{L \times d}$ is projected into queries, keys, and values:

$$Q = H^{(\ell)} W^Q, \quad K = H^{(\ell)} W^K, \quad V = H^{(\ell)} W^V, \tag{2.3}$$

where $W^Q, W^K, W^V \in \mathbb{R}^{d \times d_k}$ and d_k is the per-head key dimension. Scaled dot-product attention is then computed as

$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right), \qquad A \in \mathbb{R}^{L \times L}, \tag{2.4}$$

and the attention output is

$$H^{(\ell)}_{\mathrm{att}} = AV. \tag{2.5}$$

In multi-head attention with h heads, attention is computed in parallel for each head $j = 1, \ldots, h$, and the results are concatenated and projected:

$$\mathrm{MHA}(H^{(\ell)}) = \mathrm{Concat}\left(H^{(\ell)}_{\mathrm{att},1}, \ldots, H^{(\ell)}_{\mathrm{att},h}\right) W^O, \tag{2.6}$$

where W^O is a learned output projection. A full Transformer block combines multi-head attention with residual connections and a position-wise feed-forward network (FFN):

$$\tilde{H}^{(\ell)} = H^{(\ell)} + \mathrm{MHA}(H^{(\ell)}), \tag{2.7}$$

$$H^{(\ell+1)} = \tilde{H}^{(\ell)} + \mathrm{FFN}(\tilde{H}^{(\ell)}), \tag{2.8}$$

with layer normalisation applied in either pre-norm or post-norm variants[18, 34]. Stacking L_{layers} such blocks yields final contextual representations $H^{(L_{\mathrm{layers}})}$, where each

row corresponds to a contextual embedding for one residue position.

To perform masked language modelling, let $M \subset \{1, \ldots, L\}$ denote masked positions, and let $H^{(L_{\text{layers}})}$ be the final hidden states. For each masked position $i \in M$, the model predicts a distribution over the amino acid vocabulary V using a linear projection and softmax:

$$z_i = H_i^{(L_{\text{layers}})} W_{\text{LM}} + b_{\text{LM}}, \tag{2.9}$$

$$p_\theta(x_i \mid \tilde{x}) = \text{softmax}(z_i), \tag{2.10}$$

where $W_{\text{LM}} \in \mathbb{R}^{d \times |V|}$ and $b_{\text{LM}} \in \mathbb{R}^{|V|}$ are learnable parameters. The MLM loss for a single sequence is then

$$\mathcal{L}_{\text{MLM}}(\theta) = -\sum_{i \in M} \log p_\theta(x_i \mid \tilde{x}), \tag{2.11}$$

and over a dataset $\mathcal{D}$,

$$\mathcal{L}_{\text{MLM}}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}}\, \mathbb{E}_M \left[\sum_{i \in M} \log p_\theta(x_i \mid \tilde{x}) \right]. \tag{2.12}$$

This formulation matches the general MLM objective described earlier, with the Transformer acting as the contextual encoder producing $H^{(L_{\text{layers}})}$[8, 19].

2.3.2 Computational cost and scaling

Despite their representational power, Transformer-based protein language models incur substantial computational costs. The dominant bottleneck is self-attention, which requires forming the attention matrix $A \in \mathbb{R}^{L \times L}$ and therefore scales quadratically in sequence length[33, 34]. Specifically, computing the attention logits $QK^\top$ requires $\mathcal{O}(L^2 d_k)$ operations and storing A requires $\mathcal{O}(L^2)$ memory. For long protein sequences, this quadratic scaling leads to large memory consumption and computational overhead, especially when stacking many layers and using multiple heads.

Let L denote sequence length, d the model dimension, h the number of heads, and L_{layers} the number of layers. Up to constant factors, the attention component has approximate per-forward-pass complexity

$$\text{Cost}_{\text{attn}} \approx \mathcal{O}\Big(L_{\text{layers}} \cdot h \cdot L^2 \cdot d_k\Big), \tag{2.13}$$

while the FFN component scales roughly as

$$\text{Cost}_{\text{FFN}} \approx \mathcal{O}\Big(L_{\text{layers}} \cdot L \cdot d^2\Big). \tag{2.14}$$

These scaling laws help explain why large ESM-style models, with billions of parameters and long input sequences, typically require multi-GPU clusters and long training

times[9, 10, 19].

2.3.3 Efficiency-aware perspective and motivation for alternatives

The computational demands of Transformers also influence architectural design choices under constrained training regimes. When training time and hardware resources are limited, increasing model size or depth can lead to diminishing returns, as deeper models may not converge effectively within the available optimisation window[11, 15]. This challenges the assumption that scaling Transformers is always beneficial and motivates careful consideration of architectural efficiency, particularly in cramming-style settings where training budgets are explicitly fixed[10, 11].

Recent work has explored strategies for training compact Transformer models more efficiently. The cramming paradigm showed that, with carefully selected architectural parameters, aggressive optimisation schedules, and efficient data pipelines, Transformer-based language models can be trained within 24 hours on a single GPU while retaining competitive performance[11]. In the protein domain, [10] demonstrated that compact ESM-style Transformers trained under cramming constraints can approach the performance of much larger pre-trained models, highlighting the practical value of efficiency-oriented training methodologies.

However, the success of crammed Transformer models does not eliminate fundamental limitations associated with attention-based architectures. Even compact Transformers retain quadratic complexity with respect to sequence length, and their efficiency gains rely heavily on highly optimised attention kernels and favourable hardware characteristics[7, 33]. These limitations motivate growing interest in alternative sequence modelling architectures that may offer improved efficiency, performance trade-offs, including modern recurrent and linear-time models[3, 12, 33].

In summary, Transformer-based protein language models such as ESM and ESM2 have played a central role in advancing the field, demonstrating the effectiveness of self-attention for capturing long-range dependencies in protein sequences and establishing scaling as a powerful driver of representation quality[19, 28]. At the same time, their substantial computational requirements particularly the quadratic cost of attention, motivate the exploration of more efficient alternatives, especially under constrained training regimes[10, 11]. This tension between representational power and computational cost provides the immediate motivation for examining modern recurrent architectures in the context of protein language modelling, as developed in the following sections[3, 12].

2.4 Efficient Training and the Cramming Paradigm

The rapid escalation in computational requirements for training large language models has prompted a growing body of research focused on efficiency-oriented training methodologies. In both natural language processing and protein language modelling, the dominant scaling paradigm traditionally assumes access to abundant computational resources, enabling performance gains through increased model size, longer training duration, and extensive hyperparameter tuning[15, 16]. However, this assumption is increasingly misaligned with the realities of many academic and industrial research environments, where access to large GPU clusters and prolonged training windows is limited. As a result, efficiency has emerged as a first-class design objective rather than a secondary optimisation concern[10, 31].

The *cramming paradigm* was introduced as a principled response to these constraints. Rather than optimising models for asymptotic performance under unconstrained compute, cramming reframes training as the problem of maximizing performance within a strict computational budget, typically 24 hours on a single GPU[10, 11]. This paradigm emphasises architectural simplicity, aggressive yet stable optimisation schedules, and carefully engineered data pipelines in order to extract maximal learning progress from a limited number of parameter updates[10, 11]. Importantly, cramming does not aim to replicate the absolute performance of very large pre-trained models, but to approach it as closely as possible under fixed and realistic resource limits.

A defining characteristic of cramming is its focus on *effective learning per unit time*. Under cramming constraints, models cannot rely on slow convergence or late-stage performance gains, as training terminates once the wall-clock budget is exhausted. Consequently, architectures that exhibit stable and rapid early convergence are favoured over those that require extended training to realize their full representational capacity[11, 18]. This shift fundamentally alters the relative importance of architectural features, elevating optimisation stability, gradient flow, kernel efficiency, and memory behaviour to central considerations in model design[7, 11].

The cramming paradigm was initially developed and validated in the context of natural language processing[11], which demonstrated that Transformer-based language models trained under a 24-hour single-GPU budget can achieve performance comparable to substantially larger pre-trained counterparts by carefully tuning architectural depth and width, learning-rate schedules, and data throughput[11]. These findings challenged the assumption that large-scale pre-training is a prerequisite for high-quality language representations and highlighted the role of architectural efficiency and optimisation design in constrained training regimes[11, 15].

Subsequent work extended the cramming paradigm to protein language modelling. Frey et al. (2024) showed that compact Transformer-based protein language models

trained from scratch on UniRef50 within a fixed 24-hour budget can approach the performance of much larger pre-trained models on both upstream and downstream benchmarks. This result is particularly notable given the differences between natural language and protein sequences, including vocabulary structure, sequence length distributions, and biological constraints[10, 19]. The success of cramming in the protein domain therefore establishes that efficiency-oriented training strategies can be effective beyond textual data.

Crucially, the cramming paradigm fundamentally changes the architectural optimisation landscape. Under unconstrained training, suboptimal architectural choices can often be compensated through longer training or additional compute. Under cramming constraints, such compensation is impossible. Architectural inefficiencies directly translate into lost learning opportunities, as slower convergence or unstable training reduces the effective number of useful parameter updates achievable within the fixed time budget[10, 11]. As a result, relative model performance becomes highly sensitive to training dynamics rather than solely to theoretical expressivity.

This sensitivity has direct implications for architectural scaling. Scaling laws derived under large-compute assumptions may not hold in constrained regimes, as deeper or wider models may fail to converge sufficiently within the available training window[15, 16]. In cramming-style training, architectural depth and width must be balanced not only to maximise representational capacity but also to ensure that this capacity can be effectively trained within the fixed computational budget. This motivates systematic investigation of depth–width trade-offs, particularly for architectures whose scaling behaviour is not yet well understood.

While cramming-based studies have primarily focused on Transformer architectures, the paradigm itself is architecture-agnostic. Its principles apply equally to alternative sequence modelling approaches, including recurrent and linear-time architectures. However, the interaction between cramming constraints and architectural inductive bias may differ substantially across model families. Transformers benefit from highly optimised attention kernels and parallel computation, whereas recurrent models rely on different mechanisms for efficiency, memory usage, and optimisation stability[3, 7]. Understanding how these differences influence training outcomes under cramming constraints is therefore essential for extending the paradigm beyond attention-based models[10, 12].

In this context, the cramming paradigm provides both a motivation and a methodological framework for exploring modern recurrent architectures in protein language modelling. By imposing a strict and uniform training budget, cramming enables fair comparison between architectures with fundamentally different computational profiles. It exposes trade-offs that may be obscured under unconstrained training and highlights architectural features that promote stable and efficient learning under realistic resource

limitations[10, 12]. This perspective directly motivates the focus of the present study and sets the stage for examining modern recurrent neural networks and their variants in the following sections.

2.5 Recurrent Neural Networks in Protein Language Modelling

Recurrent neural networks (RNNs) represent the earliest generation of neural architectures applied to biological sequence modelling and played an important historical role in the development of protein language modelling. By processing sequences token by token and maintaining a hidden state that summarizes past context, recurrent models offered a natural inductive bias for proteins, which are inherently sequential polymers synthesized residue by residue. Early protein modelling approaches therefore employed vanilla RNNs as well as gated variants such as long short-term memory networks (LSTMs)[14] and gated recurrent units (GRUs) to model amino acid sequences as temporal processes[1, 27].

Formally, given a protein sequence

$$x = (x_1, \ldots, x_L),$$

a recurrent encoder updates a hidden state h_t along the sequence according to

$$h_t = f_\theta(h_{t-1}, x_t), \quad t = 1, \ldots, L,$$

where f_θ denotes the recurrent update function, such as an LSTM or GRU cell with parameters θ. This recurrence yields linear-time processing with respect to sequence length and aligns conceptually with early intuitions about protein sequence modelling.

Despite this conceptual appeal, classical recurrent architectures encountered substantial limitations as protein language modelling began to scale. A primary challenge was their difficulty in capturing long-range dependencies[14, 27]. In proteins, residues that are distant in the primary sequence may interact closely in three-dimensional space and jointly determine structure and function. Although LSTM and GRU architectures introduced gating mechanisms to mitigate vanishing and exploding gradients[14], empirical studies showed that propagating information across hundreds of sequences remained difficult in practice, particularly for long protein sequences[27, 28].

A second major limitation arose from the strictly sequential nature of classical recurrent computation. During training, the hidden state at position t depends on all previous updates, preventing parallelisation across sequence positions[7, 34]. This severely limited training computational efficiency on modern parallel hardware such as GPUs. As protein language modelling scaled to millions of sequences, this sequential dependency became increasingly prohibitive, making large-scale training inefficient

despite the nominal $\mathcal{O}(L)$ time complexity of recurrent updates[7, 34].

Optimisation difficulties further compounded these issues. Classical recurrent models are sensitive to initialisation, learning-rate schedules, and sequence length, and often exhibit unstable training dynamics when scaled to deeper architectures or large datasets[18, 25]. In protein language modelling settings, where sequences are long and corpora are large, these instabilities limited the effective depth and representational capacity of recurrent models relative to attention-based alternatives[27, 28].

In addition to architectural and computational challenges, classical recurrent models were typically trained using a causal language modelling (CLM) objective. Under CLM, the model factorizes the sequence probability as[4]

$$p_\theta(x_1, \ldots, x_L) = \prod_{t=1}^{L} p_\theta(x_t \mid x_{<t}),$$

restricting predictions to depend only on past context. While this objective is well suited to autoregressive generation, it limits access to bidirectional context and reduces the quality of computational-level representations for proteins. In contrast, masked language modelling objectives used in Transformer-based protein language models condition on both left and right context, providing richer training signals and contributing to the superior performance of attention-based models on many protein tasks[8].

Taken together, limited long-range dependency modelling, inefficient parallelisation, optimisation instability, and unidirectional training objectives. These factors led classical recurrent architectures to be gradually displaced by Transformer-based models in protein language modelling[3]. Empirical comparisons consistently showed lower perplexity and weaker downstream performance for recurrent models when trained at scale, reinforcing the dominance of attention-based architectures[22, 27, 28].

Importantly, this historical decline should not be interpreted as a fundamental limitation of recurrence as a modelling principle. Rather, it reflects the constraints of early recurrent formulations and training methodologies. Many of the challenges faced by classical RNNs stem from specific design choices, such as purely sequential updates and limited memory representations rather than from recurrence itself[3]. This distinction has motivated renewed interest in modern recurrent architectures that retain linear-time properties while addressing the expressivity, stability, and efficiency limitations of earlier models.

Recent advances in sequence modelling, including state-space models, gated linear recurrence, and hybrid architectures, demonstrate that recurrent inductive biases can be combined with structured state representations and parallelisable computation[3, 12, 26]. These developments challenge the view that attention mechanisms are the only viable foundation for large-scale sequence modelling and suggest that recurrence can again become competitive under appropriate architectural and optimisation choices.

In the context of efficiency-oriented training paradigms such as cramming, revisiting recurrent architectures becomes particularly compelling[3, 10, 11]. By combining linear-time sequence processing with improved optimisation behaviour, modern recurrent models offer a promising alternative for protein language modelling under strict computational budgets. This perspective directly motivates the focus of the present study on mLSTM-based protein language models, which aim to reconcile the efficiency advantages of recurrence with the representational demands of large-scale protein sequence modelling.

2.6 Modern Recurrent Architectures and the xLSTM Family

Recent years have witnessed a renewed interest in recurrent sequence modelling driven by the recognition that many limitations of classical recurrent neural networks arose from architectural and computational constraints rather than from recurrence itself[3, 14]. Advances in gating mechanisms, memory representations, and parallelisable computation have enabled modern recurrent architectures to address long-standing issues related to expressivity, stability, and training efficiency[3, 12, 26]. These developments challenge the dominance of attention-based models and reopen the question of whether recurrence can serve as a competitive backbone for large-scale sequence modelling, including in protein language modelling contexts.

One influential line of work in this resurgence is the *xLSTM* family of architectures. The xLSTM framework generalizes classical LSTM designs by introducing structured gating and matrix-based memory representations that enhance expressivity while preserving linear-time complexity[3]. Unlike traditional LSTMs[14], which maintain vector-valued cell states, xLSTM architectures employ matrix-valued state representations that enable richer feature interactions without requiring explicit pairwise computation across sequence positions[3].

2.6.1 Classical LSTM: Vector-valued memory

Classical LSTM architectures maintain a vector-valued cell state $c_t \in \mathbb{R}^d$ and hidden state $h_t \in \mathbb{R}^d$, updated through multiplicative gating mechanisms [13]. Given input x_t at time step t, the LSTM computes forget gate f_t, input gate i_t, output gate o_t, and a candidate cell state $\tilde{c}_t$ [3]:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \tag{2.15}$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \tag{2.16}$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \tag{2.17}$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \tag{2.18}$$

where $\sigma(\cdot)$ denotes the sigmoid function and W_*, U_*, b_* are learned parameters[3]. The cell state is updated as a gated combination of the previous state and new candidate:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \tag{2.19}$$

where $\odot$ denotes element-wise multiplication. The hidden state is then produced by gating a nonlinear transformation of the cell state [3, 13]:

$$h_t = o_t \odot \tanh(c_t). \tag{2.20}$$

This formulation enables the LSTM to selectively retain or discard information across time steps[13], addressing the vanishing gradient problem that plagued earlier recurrent architectures[14].

However, the vector-valued nature of c_t fundamentally limits the model's representational capacity[3]. Information storage and retrieval are constrained to element-wise operations, preventing the model from learning structured relationships between different features within the hidden state.

2.6.2 Multiplicative LSTM: Matrix-valued memory and content-based retrieval

The multiplicative LSTM (mLSTM) addresses the representational limitations of classical LSTM by replacing the vector-valued cell state with a *matrix-valued memory* that enables content-based storage and retrieval[3]. At each time step t, the mLSTM maintains a memory matrix

$$C_t \in \mathbb{R}^{d \times d}, \tag{2.21}$$

together with an auxiliary normalisation vector

$$n_t \in \mathbb{R}^d. \tag{2.22}$$

From scalars to structured representations. Where classical LSTM stores information as a vector of scalar activations, mLSTM stores information as a matrix of pairwise feature interactions[3]. This architectural change enables attention-like content-based retrieval while maintaining linear-time complexity through matrix-vector operations.

Input projections. Given an input embedding x_t, the mLSTM computes query, key, and value projections:

$$q_t = W_q x_t, \quad k_t = W_k x_t, \quad v_t = W_v x_t, \tag{2.23}$$

analogous to attention mechanisms[34], but used for memory updates rather than cross-position interactions.

Gating mechanisms. Similar to classical LSTM[14], mLSTM employs three gates to control information flow:

$$f_t = \sigma(W_f x_t), \quad i_t = \sigma(W_i x_t), \quad o_t = \sigma(W_o x_t), \tag{2.24}$$

where f_t, i_t, and o_t denote forget, input, and output gates, respectively. Note that unlike classical LSTM (Equations 2.15–2.17)[13], these gates depend only on the current input x_t rather than the previous hidden state h_{t-1}. This simplification enables more efficient parallel computation during training[3].

Matrix memory update. The key innovation of mLSTM lies in how memory is updated. Rather than storing a scalar value per dimension (as in Equation 2.19), mLSTM stores the *outer product* of value and key vectors[3]:

$$C_t = f_t \odot C_{t-1} + i_t \odot (v_t k_t^\top). \tag{2.25}$$

Here, $v_t k_t^\top \in \mathbb{R}^{d \times d}$ represents a rank-one matrix encoding the association between the value v_t (what to store) and key k_t (how to retrieve it). The forget gate f_t and input gate i_t control this update element-wise, exactly parallel to the classical LSTM cell update in Equation 2.19.

To enable proper normalisation during retrieval, a parallel accumulator tracks the sum of gated keys[3]:

$$n_t = f_t \odot n_{t-1} + i_t \odot k_t. \tag{2.26}$$

This normalisation vector ensures numerical stability and prevents unbounded growth of retrieved values.

Content-based retrieval. To produce the hidden state, the memory is queried using the query vector q_t. The raw retrieval is computed as[3]

$$\tilde{h}_t = \frac{C_t q_t}{\max\left(|n_t^\top q_t|, 1\right)}, \tag{2.27}$$

where the denominator normalises by the accumulated attention weights (with a lower bound of 1 to prevent division instabilities). This can be understood as a content-addressable memory lookup: the query q_t retrieves a weighted combination of all previously stored values v_τ based on their similarity to corresponding keys k_τ[3].

Finally, the output gate controls which components of the retrieved memory are

exposed[3]:

$$h_t = o_t \odot \tilde{h}_t, \tag{2.28}$$

exactly parallel to the classical LSTM output mechanism in Equation 2.20.

Connection to classical LSTM. The mLSTM formulation directly generalizes classical LSTM[3, 14]:

- If we restrict C_t to be diagonal and set $k_t = q_t = \mathbf{1}$ (all ones), the matrix-vector product $C_t q_t$ reduces to reading the diagonal entries, recovering the vector-valued cell state c_t.
- The update equation (Equation 2.25) then becomes $c_t = f_t \odot c_{t-1} + i_t \odot v_t$, matching the classical LSTM update (Equation 2.19) when $v_t = \tilde{c}_t$.
- The output mechanism (Equation 2.28) is structurally identical to classical LSTM (Equation 2.20), differing only in how $\tilde{h}_t$ is computed.

Linear-time complexity. Crucially, despite using matrix-valued memory, the mLSTM maintains $\mathcal{O}(Ld^2)$ complexity for sequences of length L, identical to classical LSTM[3]. The memory update (Equation 2.25) requires computing the rank-one outer product $v_t k_t^\top$, which takes $\mathcal{O}(d^2)$ time, and element-wise operations on C_t, also $\mathcal{O}(d^2)$. The retrieval (Equation 2.27) performs a matrix-vector product $C_t q_t$ in $\mathcal{O}(d^2)$ time. Summing over all L time steps yields $\mathcal{O}(Ld^2)$ total complexity that is linear in sequence length.

This contrasts with attention mechanisms, which require $\mathcal{O}(L^2 d)$ computation to model pairwise interactions[33, 34]. The mLSTM achieves attention-like content-based retrieval through *sequential* matrix memory updates rather than *parallel* cross-position comparisons[3].

Relevance to protein language modelling. For protein sequences, the matrix memory enables the mLSTM to learn associations between amino acid patterns and their functional consequences without explicitly comparing all pairwise positions[3]. Under cramming constraints where computational efficiency is paramount, this linear-time complexity is particularly advantageous for processing long protein sequences[10, 11].

2.6.3 Chunkwise recurrence and computational structure

A defining feature of modern mLSTM implementations is the use of *chunkwise recurrence* to mitigate the sequential bottleneck of classical recurrent computation. Given a sequence of length L, the input is partitioned into chunks of size C. Within each chunk,

Table 2.1: Comparison of classical LSTM and mLSTM formulations.

Component	**Classical LSTM**	**mLSTM**
Memory state	$c_t \in \mathbb{R}^d$ (vector)	$C_t \in \mathbb{R}^{d \times d}$ (matrix)
Storage mechanism	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$	$C_t = f_t \odot C_{t-1} + i_t \odot (v_t k_t^\top)$
Retrieval mechanism	Direct: $h_t = o_t \odot \tanh(c_t)$	Query-based: $h_t = o_t \odot \frac{C_t q_t}{\max(\lvert n_t^\top q_t \rvert, 1)}$
Gates depend on	x_t and h_{t-1}	x_t only
Complexity	$\mathcal{O}(Ld^2)$	$\mathcal{O}(Ld^2)$

computations can be parallelised, while recurrent state is propagated only across chunk boundaries[3].

Under this scheme, the per-layer computational complexity during training scales approximately as

$$\mathcal{O}\Big(\frac{L}{C} \cdot d^2 \cdot C\Big) = \mathcal{O}(Ld^2),$$

while allowing significantly improved hardware utilisation relative to strictly sequential recurrence. This structure preserves linear time asymptotic complexity with respect to sequence length while narrowing the practical efficiency gap between recurrent and attention-based architectures[3, 7].

2.6.4 Compatibility with masked language modelling

mLSTM integrates naturally with masked language modelling (MLM)[3, 8]. Given a masked sequence $\tilde{x} = (\tilde{x}_1, \ldots, \tilde{x}_L)$ with masked positions $\mathcal{M}$, a bidirectional mLSTM[8] processes the sequence in both directions to produce representations $\{h_1, \ldots, h_L\}$ (Equation 2.28).

For each masked position $i \in \mathcal{M}$, amino acid prediction uses:

$$p_\theta(x_i \mid \tilde{x}) = \text{softmax}(W_{\text{LM}} h_i + b_{\text{LM}}),$$

with training objective:

$$\mathcal{L}_{\text{MLM}}(\theta) = -\sum_{i \in \mathcal{M}} \log p_\theta(x_i \mid \tilde{x}).$$

This matches the Transformer-based MLM objective[8, 19], enabling direct architectural comparison under identical training conditions.

2.6.5 Positioning within efficiency-oriented modelling

mLSTM combines matrix-based memory, content-based retrieval, and linear-time processing to address classical recurrent architecture limitations. The matrix memory $C_t \in \mathbb{R}^{d \times d}$ provides $\mathcal{O}(d^2)$ representational capacity versus $\mathcal{O}(d)$ for vector-valued

states[3], while maintaining $\mathcal{O}(Ld^2)$ complexity—linear in sequence length—compared to self-attention's $\mathcal{O}(L^2d)$ cost[3, 33, 34].

Under cramming constraints, these properties make mLSTM a viable Transformer alternative[10, 11]. However, its behaviour under limited optimisation budgets remains uncharacterised for protein language modelling[10, 11]. This thesis investigates depth-width trade-offs in mLSTM architectures, examining whether depth scaling benefits recurrent models under severe resource constraints.

2.7 Efficient Sequence Modelling Beyond Attention

The quadratic computational complexity of self-attention ($\mathcal{O}(T^2d)$) has motivated research into more efficient sequence modelling architectures[33, 34]. As sequence lengths increase, attention-based models face unfavourable scaling in both memory and computation, prompting renewed interest in linear-time and sub-quadratic alternatives[7, 33].

Recent architectures employ diverse strategies: recurrent formulations with structured state updates (RetNet[37], RWKV[24]), convolutional sequence models (Hyena[26]), and state-space approaches (Mamba[12]). These models demonstrate that long-range dependencies can be modelled effectively without explicit pairwise attention through alternative mechanisms such as content-dependent gating and structured accumulation of past information[12, 26, 37]. Collectively, they establish that attention is not necessary for high-quality sequence modelling[12, 26, 33].

For protein language modelling, these developments provide conceptual validation. Protein sequences exhibit long-range dependencies and complex interaction patterns similar to natural language, yet face stricter efficiency constraints due to sequence length[19, 28]. The success of linear-time architectures in natural language tasks therefore supports the plausibility of non-attention approaches for biological sequences, even without direct architectural transfer[3, 12].

Within this landscape, mLSTM occupies a distinctive position. Unlike models that introduce entirely new paradigms, mLSTM extends the well-understood LSTM foundation[14] with matrix-valued memory and content-based retrieval (Section 2.6.2)[3], whilst maintaining linear-time complexity through sequential updates rather than parallel computation. This makes it particularly suitable for controlled architectural analysis under constrained training regimes[10, 11], providing a principled recurrent baseline for studying efficiency–performance trade-offs in protein language modelling.

2.8 Depth–Width Trade-Offs Under Fixed Budgets

Architectural scaling plays a central role in neural language model performance. For Transformers, extensive empirical work demonstrates that balanced increases in depth,

width, and training data yield predictable improvements in loss and downstream performance when computational resources are abundant[15, 16]. These scaling laws have strongly influenced model design in both natural language processing and protein language modelling, encouraging large architectures trained over extended periods. However, they assume unlimited compute an assumption that fails under constrained training regimes such as cramming[10, 11].

In Transformers, depth and width serve complementary functions: depth enables hierarchical refinement of representations across layers, whilst width expands representational capacity within each layer[16, 34]. Empirical studies suggest deeper Transformers capture more complex dependency structures, but excessive depth introduces optimisation difficulties and diminishing returns, particularly under limited training budgets[33, 35].

Under cramming constraints, the depth–width trade-off intensifies. Fixed training time and hardware resources mean deeper models incur higher computational cost per optimisation step, reducing the total number of updates achievable within the training window[10, 11]. Theoretical benefits of increased depth may therefore be offset by slower convergence or optimisation instability, yielding inferior final performance despite greater representational capacity[11, 18].

For recurrent architectures, depth–width trade-offs differ fundamentally. Depth lengthens optimisation paths across both time and layers, compounding gradient propagation challenges[14, 25]. Although modern recurrent architectures incorporate gating mechanisms to mitigate vanishing gradients[3, 14], deep recurrent stacks remain sensitive to reduced width and aggressive optimisation schedules under fixed parameter budgets. Increasing depth in mLSTM may therefore introduce optimisation challenges not captured by Transformer-centric scaling intuitions[3, 16].

Width plays a critical role by determining hidden state capacity and the strength of gradient signals propagated through time. Wider layers promote training stability and faster early convergence properties, especially valuable under cramming constraints[11, 18, 25]. However, allocating excessive width to shallow architectures may limit hierarchical abstraction, underscoring the need for balanced design[16].

Existing scaling laws provide limited guidance for modern recurrent architectures under strict computational budgets. Differences in computation patterns, memory access, and inductive biases mean that Transformer scaling behaviour does not transfer directly to recurrent models such as mLSTM[3, 12, 26]. This motivates dedicated empirical investigation of depth–width trade-offs in modern recurrent protein language models under constrained training regimes.

2.9 Summary and Research Gap

Protein language modelling has advanced rapidly through large-scale sequence data, self-supervised learning, and powerful sequence modelling architectures. Transformer-based models, particularly ESM and ESM2, achieve strong empirical performance by leveraging global self-attention to capture long-range dependencies[19, 28], but impose substantial computational demands[9, 19].

The cramming paradigm reframes training as optimisation under fixed, realistic computational budgets. By emphasising architectural efficiency, stable optimisation dynamics, and effective learning per unit time, cramming demonstrates that compact models trained from scratch can approach the performance of much larger pre-trained models[10, 11]. However, existing cramming studies focus almost exclusively on Transformers, benefiting from mature implementations and well-characterised scaling behaviour[10, 11].

Modern recurrent architectures provide strong motivation to revisit recurrence for protein language modelling. mLSTM introduces matrix-based memory, content-based retrieval, and linear-time complexity that address limitations of classical recurrent networks[14] whilst avoiding attention's quadratic cost (Section 2.6.2)[3]. Parallel advances in state-space and hybrid models further demonstrate that attention is not necessary for effective long-range sequence modelling under efficiency constraints[12, 26].

Despite these advances, the literature offers limited empirical guidance on configuring modern recurrent protein language models under fixed computational budgets. In particular, there exists no systematic study of depth–width trade-offs in mLSTM-based protein language models trained under cramming constraints[3, 10, 11]. Scaling laws derived for Transformers provide limited insight into recurrent architectures due to fundamental differences in computation, optimisation dynamics, and inductive biases[15, 16].

This gap is amplified by evaluation practices. Downstream benchmarks introduce confounding factors that obscure architectural effects under constrained budgets, whilst upstream evaluation using training dynamics and validation perplexity offers a more controlled lens[10, 11]. Few studies focus exclusively on upstream metrics when comparing recurrent and attention-based models under identical constraints[10, 11].

Research gap: There exists no systematic empirical investigation of depth–width trade-offs in modern recurrent protein language models trained under strict cramming constraints. Addressing this gap is essential for principled architectural design in efficiency-oriented protein language modelling and for extending the cramming paradigm beyond Transformer-centric approaches[3, 10, 11].

This thesis directly addresses this gap through controlled experimental analysis of mLSTM-based protein language models with varying architectural depth under fixed

parameter budgets[3, 10, 11], establishing the foundation for the methodology presented in Chapter 3.

3 Methodology

This chapter presents the experimental methodology used to investigate architectural depth–width trade-offs in mLSTM-based[3] protein language models trained under cramming-style computational constraints[10, 11]. The methodology is designed to evaluate both the upstream representation learning behaviour and the downstream utility of compact protein language models trained under fixed time, memory, and parameter budgets.

The study comprises two tightly connected experimental components. The *primary* component focuses on upstream masked language modelling (MLM)[8] on protein sequences drawn from the UniRef50 dataset[2, 32], where architectural and optimisation effects can be analysed in a controlled and intrinsic setting. This includes (i) a direct architectural comparison between a 6-layer mLSTM model and a 6-layer ESM2-style Transformer baseline[19] under matched parameter budgets, and (ii) a systematic depth scaling analysis of mLSTM variants with 1, 6, and 12 layers while maintaining approximate parameter parity.

The *secondary* component extends the analysis to downstream protein property prediction tasks. Representations learned during upstream pre-training are evaluated on established downstream benchmarks[6] to assess whether differences observed at the MLM level translate to functional performance differences. This downstream evaluation provides an additional perspective on representation quality that is not captured by token-level likelihood metrics alone.

The chapter is organised as follows. Section 3.1 describes the overall methodological approach and study design. Section 3.2 details the dataset acquisition and preprocessing pipeline. Section 3.3 describes the tokenisation and input representation strategy for protein sequences. Section 3.4 formalises the masked language modelling objective used for upstream training. Section 3.9 presents the complete experimental design, covering both the baseline comparison and mLSTM depth exploration. Section 3.10 specifies the model architectures and parameter configurations. Section 3.12 details the training configuration and hyperparameter settings. Section 3.14 defines both upstream and downstream evaluation metrics. Section 3.13 provides a formal algorithmic description of the training procedure. Finally, Section 3.18 summarises the methodological framework.

Scope Clarification: The primary emphasis of this methodology is on upstream masked language modelling under cramming-style constraints[10, 11], which forms the basis for all architectural and optimisation comparisons. Downstream task evaluation[6] is included as a complementary analysis to assess the transferability of learned protein

sequence representations. The downstream experiments do not involve task-specific fine-tuning of model parameters; instead, fixed representations extracted from pre-trained checkpoints are evaluated[10] to isolate the effect of upstream training and architecture on downstream performance.

3.1 Methodological Approach and Experimental Design

This research adopts a quantitative, controlled experimental methodology to investigate architectural design choices in compact protein language models trained under cramming-style computational constraints[10, 11]. The study is situated within the cramming paradigm, in which models are trained from scratch under strict limits on wall-clock time, memory, and parameter count, using a single GPU[10, 11].

The overarching methodological goal is to understand how architectural inductive biases, specifically attention-based[34] versus recurrent formulations[3] and architectural depth–width trade-offs[16] influence both upstream masked language modelling behaviour and the downstream utility of learned protein representations when training resources are severely constrained.

To achieve this goal, the methodology is structured as a two-stage experimental study. The primary stage focuses on upstream masked language modelling (MLM)[8] on protein sequences drawn from the UniRef50 dataset[2, 32], enabling intrinsic analysis of optimisation behaviour, convergence dynamics, and representation learning under controlled conditions. The secondary stage evaluates the downstream transferability of the learned representations on established protein property prediction benchmarks[6], providing a complementary assessment of practical utility beyond token-level likelihood metrics.

Across all experiments, architectural configuration is treated as the primary independent variable. All other factors including dataset, preprocessing pipeline, training budget, optimisation schedule, and hardware environment are held constant. Dependent variables include upstream performance metrics for validation loss and validation perplexity, training efficiency indicators for convergence speed and stability, and downstream evaluation metrics that is, the ranked correlation and regression error.

3.1.1 Research Design Overview

The experimental design consists of three tightly connected components:

- Upstream Baseline Architectural Comparison: A direct comparison between a 6-layer mLSTM model[3] and a 6-layer ESM2-style Transformer baseline[19]. Both models are parameter-matched at approximately 8 million parameters and trained under identical cramming-style constraints[10, 11]. This comparison establishes a reference point for assessing whether modern recurrent architectures

can achieve comparable upstream protein MLM performance to Transformer-based models within a fixed computational budget.

- Upstream Depth Scaling Analysis: A systematic investigation of architectural depth in mLSTM-based protein language models[3]. Variants with 1 layer (shallow–wide), 6 layers (balanced), and 12 layers (deep–narrow) are trained while maintaining approximate parameter parity[16]. This design isolates the effect of depth allocation on convergence behaviour, training stability, and upstream MLM performance.

- Downstream Representation Evaluation: Representations extracted from the final pre-trained checkpoints are evaluated on downstream protein property prediction tasks from the FLIP benchmark[6]. These evaluations assess whether differences observed during upstream MLM training translate into differences in functional prediction performance. No task-specific fine-tuning is performed; downstream models operate on fixed embeddings[10, 28] to isolate the effect of upstream training and architecture.

The ESM2-style Transformer is treated as a fixed reference baseline and is not subjected to depth variation. This design choice reflects prior evidence that Transformer scaling behaviour under cramming constraints has been characterised in earlier work[10, 11], and it enables focused analysis of depth–width trade-offs within modern recurrent architectures under practical computational limits.

3.1.2 Research Objectives

The experimental design directly supports the Research Objectives (RO) and questions defined in Chapter 1.

- RO1, Architectural Comparison: To evaluate whether an mLSTM-based protein language model can achieve competitive upstream masked language modelling performance relative to an ESM2-style Transformer when both are trained from scratch under identical cramming-style constraints and matched parameter budgets. This objective is addressed through the baseline architectural comparison.

- RO2, Depth Scaling Effects: To investigate how architectural depth affects upstream protein MLM performance, convergence behaviour, and training stability in mLSTM models when the total parameter count and training budget are fixed. This objective is addressed through the systematic depth scaling analysis.

- RO3, Training Efficiency and Stability: To analyse how architectural choices influence optimisation efficiency under constrained training, as reflected in loss

trajectories, convergence speed, and sensitivity to learning-rate warm-up schedules. This objective is addressed through comparative analysis of training curves and hyperparameter sensitivity.

- RO4, Downstream Transferability: To assess whether representations learned under cramming-style constraints retain utility for downstream protein property prediction tasks from the FLIP benchmark, and whether architectural differences observed upstream are reflected in downstream performance. This objective is addressed through downstream evaluation using fixed pre-trained embeddings.

3.1.3 Experimental Constraints and Justification

All experiments are conducted under a common set of constraints motivated by the cramming paradigm[10, 11]:

- Bias-Free Linear Layers: Bias terms are removed from linear transformations in both the ESM2 baseline and the mLSTM models. This design choice follows efficiency-oriented practices in cramming-style training, where bias parameters contribute marginally to representational capacity while incurring additional computation[10, 11]. Applying this modification uniformly ensures architectural comparability under the same compute budget.

- Fixed Training Budget: All models are trained for exactly 10,000 optimisation steps, corresponding to approximately 24 hours of training on a single NVIDIA Tesla V100-PCIE-16GB GPU.

- Single-GPU Training: Training is restricted to a single GPU to reflect realistic academic resource constraints and to avoid confounding effects from distributed optimisation[11].

- Fixed Parameter Budget: All architectures are designed to contain approximately 8 million trainable parameters, enabling controlled comparisons of architectural efficiency rather than raw capacity[11, 16].

- Identical Data and Preprocessing: All models are trained on the same UniRef50 dataset[2, 32] splits using identical tokenisation, sequence length constraints, masking strategies, and data loading pipelines.

Together, these constraints ensure that observed differences in upstream and downstream performance can be attributed primarily to architectural design choices and depth allocation, rather than differences in data, compute, or training duration.

3.2 Dataset and Data Acquisition

This study uses two distinct categories of datasets corresponding to the two stages of the experimental design: (i) large-scale unlabelled protein sequence data for upstream masked language model pre-training, and (ii) labeled protein datasets for downstream evaluation of representation transferability. These datasets are used for different purposes and are never mixed during training.

3.2.1 Upstream Pre-training Dataset: UniRef50

All models are pre-trained on the UniRef50 dataset, a widely used protein sequence database clustered at 50% sequence identity [2, 32]. UniRef50 reduces redundancy while maintaining broad coverage of protein families, making it well suited for learning general-purpose protein sequence representations[19, 32].

The dataset consists exclusively of amino acid sequences without explicit functional labels. This property makes UniRef50 appropriate for self-supervised training using the masked language modelling (MLM) objective[8, 28], where models learn to reconstruct masked amino-acid tokens from surrounding sequence context. All upstream results reported in Chapter 4 are derived solely from training on UniRef50.

3.2.2 UniRef50 Access and Preprocessing Pipeline

The UniRef50 dataset was accessed in FASTA format and processed using a custom preprocessing pipeline designed for efficiency and reproducibility under constrained hardware conditions. The pipeline performs sequence filtering, tokenization, dataset splitting, and conversion into memory-efficient formats suitable for streaming during training.

To support consistent experimentation across architectures, the preprocessing pipeline produces the following artifacts:

- Filtered FASTA files containing only sequences satisfying length constraints
- Fixed training and validation splits saved as separate files
- Tokenized protein sequences streamed during training to minimise memory usage

All models use the same preprocessed files and dataset splits, ensuring that observed differences in training behaviour and performance are attributable to architectural choices rather than data variation.

3.2.3 Sequence Length Filtering

Protein sequences in UniRef50 exhibit substantial variability in length, with many sequences exceeding the maximum context length that can be processed efficiently under limited GPU memory. To ensure computational feasibility and consistent memory usage, sequences were filtered to a maximum length of $L_{\max} = 512$ tokens[10].

Sequences exceeding this threshold were excluded from both the training and validation sets. This constraint reflects practical memory limits on the NVIDIA Tesla V100-PCIE-16GB GPU and aligns with common practices in protein language modelling under resource constraints[10, 11]. All upstream training and validation results should be interpreted in the context of protein sequences bounded to this maximum length.

3.2.4 Training and Validation Split

The filtered UniRef50 dataset was partitioned into fixed training and validation subsets. The validation set is used exclusively for monitoring generalization performance during training and for computing final upstream evaluation metrics, including validation loss and validation perplexity[10, 11].

The split is held constant across all experiments to ensure fair and directly comparable evaluation across architectures, depth configurations, and hyperparameter settings.

3.2.5 Downstream Evaluation Dataset: FLIP Benchmark

To evaluate the downstream utility of representations learned during upstream pre-training, this study uses the FLIP (Fitness Landscape Inference for Proteins) benchmark[6]. FLIP is a standardized collection of protein property prediction tasks designed to assess how well pre-trained protein language models transfer to supervised prediction problems.

Importantly, FLIP datasets are *not* used during pre-training. They are employed exclusively for downstream evaluation using frozen representations extracted from the final pre-trained checkpoints[10, 28].

3.2.6 FLIP Tasks Used in This Study

Three FLIP benchmark tasks[6] are considered, covering diverse biological properties and sequence-function relationships:

- Adeno-Associated Virus fitness (AAV): Predicts viral capsid fitness based on amino-acid sequence variations[6]. This task is characterised by strong epistatic effects and long-range sequence dependencies, making it a challenging benchmark for representation quality.

- Protein G domain binding affinity (GB1): A mutational scanning task that evaluates the ability to predict binding affinity changes resulting from amino-acid substitutions[6]. This task emphasises local sequence structure relationships.
- Thermostability prediction (Meltome): Predicts protein melting temperature and thermostability from sequence information[6], capturing global biophysical properties related to folding and stability.

These tasks collectively probe different aspects of protein sequence understanding, including local residue interactions, global structural constraints, and evolutionary fitness landscapes.

3.2.7 Downstream Data Usage Protocol

For downstream evaluation, pre-trained models are used as fixed feature extractors[10, 28]. Sequence embeddings are obtained from the final pre-trained checkpoint without any additional fine-tuning of model parameters. These embeddings are then used as inputs to simple supervised regression models trained on the labelled FLIP datasets[6].

Performance is evaluated using task-appropriate metrics, including Spearman's rank correlation coefficient and mean squared error (MSE)[6]. This protocol isolates the effect of upstream representation learning and architectural choice, ensuring that downstream performance differences reflect representation quality rather than task-specific fine-tuning.

3.3 Tokenization and Input Representation

Protein sequences are tokenized at the residue level, mapping each of the 20 standard amino acids to unique integer indices. The vocabulary is augmented with special tokens for padding, masking, and sequence boundaries.

For the mLSTM architecture, a minimal vocabulary of 23 tokens is used, consisting of 20 amino acids and 3 special tokens. For the ESM2-style Transformer, a vocabulary of 33 tokens is used, following the ESM2 tokenization scheme[19] to maintain architectural compatibility and allow future extensions to transfer learning.

All sequences are padded or truncated to a maximum length of $L_{\max} = 512$ tokens[10]. Padding positions are explicitly masked during loss computation to ensure that they do not contribute to the training objective or evaluation metrics.

3.4 Upstream Training Objective: Masked Language Modelling

All models are pre-trained using a masked language modelling (MLM) objective[8, 28] on unlabelled protein sequences from UniRef50[2, 32]. MLM is a self-supervised

learning strategy[8] in which a model is trained to reconstruct randomly masked amino-acid residues from their surrounding sequence context.

Given a protein sequence $\mathbf{x} = (x_1, \ldots, x_L)$, where $L \leq 512$, a stochastic masking procedure is applied to generate a corrupted input sequence $\tilde{\mathbf{x}}$ and a set of masked positions $M \subset \{1, \ldots, L\}$.

3.5 Masking Strategy

Following established practice in protein language modelling[8, 19, 28], 25% of tokens in each sequence are selected for masking. For each selected position $i \in M$, the original token x_i is replaced with a special `[MASK]` token.

Formally, the corrupted sequence is constructed as:

$$\tilde{x}_i = \begin{cases} \texttt{[MASK]} & \text{if } i \in M \\ x_i & \text{otherwise.} \end{cases} \tag{3.1}$$

The model is trained to predict the original amino-acid identity at each masked position conditioned on the corrupted sequence $\tilde{\mathbf{x}}$.

3.6 Upstream Loss Function

The upstream training objective is defined as the average cross-entropy loss over masked positions. Let $f_\theta(\tilde{\mathbf{x}})$ denote the model output logits, where θ represents the model parameters. The MLM loss is defined as:

$$\mathcal{L}_{\text{MLM}}(\theta) = -\frac{1}{|M|} \sum_{i \in M} \log p_\theta(x_i \mid \tilde{\mathbf{x}}), \tag{3.2}$$

where $p_\theta(x_i \mid \tilde{\mathbf{x}})$ is the softmax-normalised probability assigned to the true amino-acid token at position i.

This loss provides a bidirectional training signal[8, 28], which encourages the model to leverage both the local and long-range sequence context. Optimisation of $\mathcal{L}_{\text{MLM}}$ constitutes the *only* training objective during pre-training. No downstream task loss is used to update model parameters.

3.7 Downstream Evaluation Tasks

In addition to upstream masked language modelling, this study evaluates the downstream utility of pre-trained representations on supervised protein property prediction tasks[6]. Downstream evaluation is performed using the FLIP benchmark[6] and is strictly separated from upstream training.

Pre-trained protein language models are used as fixed feature extractors[10, 28]. No gradients from downstream tasks are propagated into the pre-trained model parameters.

3.8 Downstream Prediction Objective

For downstream tasks, sequence-level embeddings are extracted from the final pre-trained checkpoint and used as inputs to lightweight regression models. Let $\mathbf{h}(\mathbf{x})$ denote the fixed embedding of a protein sequence $\mathbf{x}$.

For a dataset of labeled protein sequences $\{(\mathbf{x}^{(n)}, y^{(n)})\}$, downstream prediction is formulated as a regression problem. Model performance is evaluated using the following metrics:

- Spearman's rank correlation coefficient (ρ): Measures the monotonic relationship between predicted and true values, making it robust to non-linear scaling effects common in protein fitness landscapes[6].
- Mean Squared Error (MSE): Quantifies absolute prediction accuracy in the original label space[6].

These metrics are used exclusively for evaluation and comparison. They do not define training objectives for the protein language models themselves.

Downstream Evaluation Phase. Following upstream pre-training, selected model checkpoints are evaluated on downstream protein property prediction tasks using the FLIP benchmark[6]. This phase assesses the transferability of learned representations without additional fine-tuning[10, 28] and provides a complementary evaluation signal beyond intrinsic MLM metrics.

3.9 Experimental Design

The experimental design is structured to isolate the effect of architectural choices under strict computational constraints[10, 11] while maintaining reproducibility and fairness across comparisons. In particular, the design explicitly controls for parameter count, training duration, dataset composition, and optimisation strategy so that observed differences in performance can be attributed primarily to architectural structure rather than scale or compute availability.

The experiment is organised into two upstream pre-training phases followed by a downstream evaluation phase. Together, these components enable analysis of (i) intrinsic masked language modelling behaviour during pre-training and (ii) the transferability of learned protein sequence representations to downstream prediction tasks[6].

3.9.1 Phase 1: Baseline Architectural Comparison

The first experimental phase compares a 6-layer mLSTM model[3] against a 6-layer ESM2-style Transformer[19] under matched parameter budgets and identical training constraints[10, 11]. Both models contain approximately 8 million trainable parameters and are trained using the same dataset, preprocessing pipeline, batch size, and optimisation schedule.

This comparison establishes whether modern recurrent architectures (mLSTM)[3] can achieve competitive upstream masked language modelling performance relative to Transformer-based models[19] when both are trained from scratch under cramming-style constraints[10, 11]. The 6-layer configuration represents a balanced trade-off between depth and width within the fixed parameter budget and serves as the reference point for subsequent depth scaling experiments.

3.9.2 Phase 2: Depth Scaling Analysis

The second experimental phase investigates how architectural depth influences upstream performance and training dynamics in mLSTM-based protein language models[3]. Three depth configurations are evaluated:

- Shallow 1-layer: A single wide mLSTM layer with increased hidden dimension to match the parameter budget[16]
- Balanced 6-layer: A balanced configuration matching the baseline comparison
- Deep 12-layer: A deep narrow stack with reduced per-layer width to maintain the parameter budget[16]

For each depth configuration, the hidden dimension is adjusted such that all variants contain approximately 8 million parameters. This iso-parametric design[11, 16] isolates the effect of depth by ensuring that differences in performance cannot be attributed to differences in total model capacity.

3.9.3 Hyperparameter Exploration

To assess sensitivity to optimisation settings, each architecture and depth configuration is trained under multiple hyperparameter combinations[10, 11]. Two key hyperparameters are varied in a controlled manner:

- Learning Rate: Two learning rates are evaluated: 4×10^{-4} and 1×10^{-3}
- Warm-up Duration: Two warm-up schedules are evaluated: 1000 steps and 2000 steps[11, 18]

These settings are combined factorially, yielding four training runs per architectural depth configuration. This design enables the assessment of whether different architectures or depth configurations exhibit different sensitivities to learning rate and warm-up schedules[18].

Rather than performing exhaustive hyperparameter optimisation, this exploration is intended to assess robustness and sensitivity under realistic cramming constraints[10, 11], where extensive tuning is impractical. The selected ranges reflect commonly used settings in prior protein language modelling and cramming-based studies[10, 11].

3.9.4 Complete Experimental Matrix

Table 3.1 summarizes the complete experimental design. The study consists of 16 total training runs: 4 runs for the ESM2 baseline (1 depth $\times$ 2 learning rates $\times$ 2 warm-up schedules) and 12 runs for mLSTM variants (3 depths $\times$ 2 learning rates $\times$ 2 warm-up schedules).

Table 3.1: Complete experimental matrix showing all training configurations. Each cell represents a single training run.

Architecture	**Depth**	**Params**	**LR**	**Warmup**	**Runs**
ESM2 (Transformer)	6 layers	∼8M	4×10^{-4}	1000	1
			4×10^{-4}	2000	1
			1×10^{-3}	1000	1
			1×10^{-3}	2000	1
mLSTM	1 layer (shallow)	∼8M	4×10^{-4}	1000	1
			4×10^{-4}	2000	1
			1×10^{-3}	1000	1
			1×10^{-3}	2000	1
mLSTM	6 layers (balanced)	∼8M	4×10^{-4}	1000	1
			4×10^{-4}	2000	1
			1×10^{-3}	1000	1
			1×10^{-3}	2000	1
mLSTM	12 layers (deep)	∼8M	4×10^{-4}	1000	1
			4×10^{-4}	2000	1
			1×10^{-3}	1000	1
			1×10^{-3}	2000	1
Total					**16**

The experimental matrix reflects a deliberately constrained yet systematic design[10,

11]. Each configuration is trained exactly once, consistent with the cramming paradigm's emphasis on efficiency and rapid iteration[10, 11]. While this limits statistical averaging across random seeds, it enables direct comparison of architectural behaviour under realistic academic compute constraints and mirrors practical scenarios in which model selection must be made based on limited experimental runs.

3.10 Model Architectures

This section provides detailed architectural specifications for the ESM2-style Transformer baseline and the mLSTM depth variants. All models are designed to contain approximately 8 million trainable parameters to enable capacity-controlled comparisons[11, 16].

3.10.1 ESM2-Style Transformer Baseline

The Transformer baseline follows an ESM2-style architecture[19] adapted for training from scratch under cramming constraints[10, 11]. The model consists of 6 Transformer encoder layers[34], each containing multi-head self-attention and position-wise feed-forward sublayers with residual connections and layer normalisation[34].

The architectural hyperparameters are as follows:

- Number of Layers: 6
- Hidden Dimension: 320
- Number of Attention Heads: 20
- Attention Head Dimension: 16 (320 / 20)
- Feed-Forward Dimension: 2,048
- Vocabulary Size: 33
- Maximum Sequence Length: 512
- Bias Terms: Removed (following cramming best practices)[10, 11]
- Dropout: 0.0 (no dropout)

This configuration yields approximately 7.49 million trainable parameters. The architecture uses standard Transformer encoder blocks with Pre-LayerNorm[18, 35], GELU activations, and no dropout. Bias terms are removed from all linear layers to reduce parameter count and improve memory efficiency, following recommendations from the cramming literature[10, 11].

3.10.2 mLSTM Architecture Variants

The mLSTM models are based on the multiplicative Long Short-Term Memory architecture introduced in the xLSTM family[3]. The architecture employs exponential gating, matrix-based memory, and chunk-wise recurrent operations[3] to enable efficient sequence modelling with linear time complexity.

3.10.3 Bidirectional Processing Adaptation

Standard mLSTM architectures are designed for causal language modelling and process sequences unidirectionally[3]. To enable fair comparison with the bidirectional ESM2 Transformer under the MLM objective[8, 19], the mLSTM model is adapted to incorporate bidirectional context.

The bidirectional adaptation is implemented by concatenating forward and reversed sequences into a single batch of size, processing both directions through the same mLSTM kernel in a single pass, and combining the forward and backward hidden states through element-wise summation after reversing the backward sequence to match the original sequence order. This approach provides bidirectional representations suitable for masked token prediction[3, 8] while maintaining the $O(n)$ time complexity of the recurrent architecture.

3.10.4 Depth Variant Specifications

Three mLSTM depth configurations are evaluated, with hidden dimensions adjusted to maintain approximately 8 million parameters across all variants[11, 16]. The architectural hyperparameters for each configuration are summarized in Table 3.2.

Table 3.2: mLSTM architectural configurations for depth scaling analysis. Hidden dimensions are adjusted to maintain approximately 8M parameters across all depth variants.

Configuration	Layers	Hidden Dim	Heads	Head Dim	Params
Shallow (1-layer)	1	784	8	98	∼8M
Balanced (6-layer)	6	320	8	40	∼8M
Deep (12-layer)	12	226	8	28	∼8M

All mLSTM variants share the following architectural settings:

- Number of Layers: 6
- Hidden Dimension: 320
- Number of Attention Heads: 8

- Attention Head Dimension: 40 (320 / 8)
- Vocabulary Size: 23
- Maximum Sequence Length: 512
- Maximum Position Embeddings: 2,048
- Bidirectional Processing: Enabled[8]
- Bias Terms: Removed[10, 11]
- Dropout: 0.0

To isolate the effect of depth, all models are constrained to approximately 8M parameters[11, 16]. Shallow models achieve this parameter budget by using fewer but wider layers, allocating more capacity per layer. Deep models achieve the same budget by using more but narrower layers, distributing capacity across depth. This parameter matching ensures that performance differences reflect architectural depth rather than model size[16].

3.11 Training Paradigm: Cramming-Style Optimisation

This study adopts the *cramming* training paradigm, an efficiency-oriented approach to neural language model training introduced by Geiping et al.[11] and further explored by Frey et al.[10] in the context of protein language modelling. Cramming-style training is defined not by a specific model architecture or optimiser, but by a strict set of computational constraints and design principles aimed at maximising learning progress within a severely limited training budget.

3.11.1 Cramming-Style Training

Cramming-style training refers to training neural language models *from scratch* under a fixed and small computational budget, typically on the order of hours rather than days or weeks[10, 11]. In contrast to conventional large-scale pre-training, which relies on extensive compute, long training horizons, and large parameter counts[9, 16, 19], cramming explicitly restricts training time, hardware resources, and optimisation steps.

Formally, a training setup is considered *cramming-style* if it satisfies the following conditions:

- Fixed compute budget: Training is limited to a predefined number of optimisation steps and wall-clock time, with no extension based on convergence[10, 11].

- Training from scratch: Model parameters are randomly initialized, without the use of pre-trained weights or transfer learning during pre-training[11].
- Single-device execution: Training is performed on a single GPU with constrained memory, reflecting realistic academic or resource-limited settings[10, 11].
- Efficiency-oriented design: Architectural and optimisation choices are selected to maximize learning efficiency per update, rather than asymptotic performance[10, 11].

Within this paradigm, performance is evaluated relative to the imposed constraints, emphasising how effectively different architectures convert a limited training budget into useful representations[10, 11].

3.11.2 Implications for Model Training and Evaluation

Under cramming-style constraints, models are not expected to reach the same level of performance as fully pre-trained counterparts trained for hundreds of thousands of updates[9, 19]. Instead, the objective is to compare architectures based on their *learning efficiency*, measured by validation loss reduction, convergence behaviour, and downstream utility achieved within the fixed budget[10, 11].

This perspective is particularly relevant for protein language modelling, where large pre-trained models are computationally expensive and often inaccessible[9, 19]. Cramming-style training enables systematic investigation of architectural design choices under realistic resource constraints[10, 11], providing insights into which inductive biases are most effective when optimisation time is limited.

3.11.3 Relation to the Training Algorithm

Algorithm 1 instantiates the cramming paradigm in the context of protein masked language modelling[8, 28]. The algorithm combines gradient accumulation to achieve a large effective batch size, a linear warm-up and decay learning-rate schedule[11, 18], and gradient clipping[25] to ensure numerical stability. Logging, validation, and checkpoint saving are integrated directly into the training loop to support monitoring under the fixed training budget[10, 11].

Together, these components operationalize cramming-style training by ensuring that each optimisation step contributes maximally to learning progress within the imposed computational constraints[10, 11].

3.12 Training Configuration

This section details the training configuration, including hardware environment, batching strategy, optimisation settings, and learning rate schedules. All settings are held constant across experiments except for the controlled variation of learning rate and warm-up duration.

3.12.1 Hardware Environment and Computational Constraints

All experiments are conducted using a single NVIDIA Tesla V100-PCIE-16GB GPU, with 16 GB of GPU memory[10, 11]. Training is performed under strict memory constraints consistent with the cramming paradigm. The hardware environment reflects realistic resource limitations commonly encountered in academic settings and enables fair comparison between architectures under identical computational conditions.

3.12.2 Training Duration and Budget

All models are trained for exactly 10,000 optimisation updates, corresponding to approximately 24 hours of wall-clock training time on the V100 GPU[10, 11]. The fixed update budget ensures that all models receive identical optimisation opportunities regardless of per-step computational cost.

3.12.3 Batching and Gradient Accumulation

Due to GPU memory constraints, training uses a physical batch size of 12 sequences per forward-backward pass. Gradient accumulation is applied over 24 steps to achieve a larger effective batch size without exceeding memory limits[11]. The effective batch size is therefore:

$$B_{\text{eff}} = B_{\text{phys}} \times G_{\text{accum}} = 12 \times 24 = 288 \text{ sequences} \tag{3.3}$$

where B_{phys} is the physical batch size and G_{accum} is the number of gradient accumulation steps.

This batching strategy improves optimisation stability by reducing gradient variance while maintaining compatibility with the GPU memory constraints[10, 11]. Gradient updates are applied to model parameters every 24 forward-backward passes.

3.12.4 Optimisation Settings

All models are trained using the AdamW optimiser[20] with the following hyperparameters:

- Learning Rate: $\eta \in \{4 \times 10^{-4}, 1 \times 10^{-3}\}$ (varied per experiment)

- Beta Coefficients: $\beta_1 = 0.99$, $\beta_2 = 0.98$
- Epsilon: $\epsilon = 10^{-12}$
- Weight Decay: $\lambda = 0.0$ (no weight decay)
- Gradient Clipping: Maximum gradient norm of 0.5[25]

The beta coefficients and epsilon value follow recommendations from the protein cramming literature[10]. Gradient clipping is applied to prevent gradient explosion and improve training stability[25], particularly important for deeper architectures.

3.12.5 Learning Rate Schedule

Training uses a linear warm-up followed by linear decay schedule[11, 18]. The learning rate increases linearly from 0 to the maximum learning rate $\eta_{\max}$ over the first W warm-up steps, then decays linearly to 0 over the remaining training steps.

Formally, the learning rate at step t is:

$$\eta(t) = \begin{cases} \eta_{\max} \cdot \frac{t}{W} & \text{if } t \leq W \\ \eta_{\max} \cdot \frac{T-t}{T-W} & \text{if } t > W \end{cases} \tag{3.4}$$

where $W \in \{1000, 2000\}$ is the warm-up duration varied per experiment and $T =$ 10,000 is the total number of training steps.

The warm-up phase stabilises the early training dynamics by preventing large parameter updates before the optimiser has accumulated sufficient gradient statistics[11, 18]. The controlled variation of warm-up duration enables assessment of sensitivity to this hyperparameter across architectures and depth configurations.

3.12.6 Training Stability Mechanisms

To improve training stability under cramming constraints[10, 11], the following mechanisms are employed:

- Mixed Precision Training: FP16 half-precision[23] is used for forward and backward passes to reduce memory usage and accelerate computation, with FP32 full-precision maintained for parameter updates.
- Gradient Clipping: Clipping of the global gradient norm at 0.5[25] prevents gradient explosion.
- Pre-LayerNorm: Layer normalisation is applied before attention and feed-forward sublayers rather than after[18, 35], improving gradient flow in deeper architectures.

3.12.7 Checkpoint and Logging Configuration

Model checkpoints are saved at the following optimisation steps: 100, 500, 1000, 5000, 9000, and 10,000. The final checkpoints at step 10,000 are used for the upstream evaluation. Training and validation metrics are logged every 10 optimisation steps. Validation is performed every 500 training steps by evaluating the model on 100 validation batches.

3.13 Training Procedure and Algorithm

This section provides a formal algorithmic description of the training procedure used across all experiments. The training procedure is identical for both the Transformer baseline and the mLSTM variants to ensure experimental comparability.

Training Workflow The training procedure consists of the following stages:

1. Data Loading and Preprocessing: Load tokenized protein sequences from the filtered training dataset[2, 32]. Apply sequence length truncation and padding to the maximum length of 512 tokens.

2. Masking: Apply the masked language modelling corruption strategy[8] to 25% of tokens in each sequence, producing corrupted inputs $\tilde{\mathbf{x}}$ and target labels $\mathbf{y}$.

3. Forward Pass: Pass the corrupted input through the model to obtain logits for all positions.

4. Loss Computation: Compute the MLM loss[8, 28] by comparing model logits to target labels at masked positions only.

5. Backward Pass: Backpropagate gradients through the model.

6. Gradient Accumulation: Accumulate gradients over 24 forward-backward passes to form an effective batch[11].

7. Parameter Update: Apply gradient clipping[25], update model parameters using AdamW[20], and zero accumulated gradients.

8. Learning Rate Schedule: Update the learning rate according to the warm-up and decay schedule[11, 18].

9. Validation: Periodically evaluate the model on the validation set to compute validation loss and perplexity[10, 11].

10. Checkpointing: Save model checkpoints at predetermined steps.

3.13.1 Training Algorithm

Algorithm 1 provides a formal pseudocode description of the cramming-style training procedure[10, 11] used for protein masked language modelling (MLM)[8, 28]. The procedure performs gradient accumulation to achieve a larger effective batch size, followed by a parameter update with gradient clipping[25] and a warm-up/decay learning-rate schedule[11, 18]. Training progress is monitored through periodic logging, validation, and checkpoints.

3.13.2 Implementation Details

The training procedure is implemented in Python using PyTorch. Data loading uses custom streaming data loaders to efficiently handle large protein sequence datasets without loading the entire dataset into memory. Mixed precision training[23] is implemented using PyTorch's automatic mixed precision (AMP) with gradient scaling to maintain numerical stability.

The mLSTM architecture is implemented using custom CUDA kernels for efficient chunkwise recurrent operations[3, 7]. The ESM2-style Transformer uses PyTorch's native multi-head attention implementation with Flash Attention optimisations where available[7].

3.14 Evaluation Metrics and Performance Assessment

This section defines the upstream evaluation metrics used to assess model performance and training behaviour. All evaluation is performed only on upstream masked language modelling performance. Downstream task evaluation on protein function prediction benchmarks is outside the scope of this thesis.

3.14.1 Training and Validation Loss

The training loss is computed as the mean MLM loss over the training batches and is logged at regular intervals throughout training. Training loss curves provide insight into the optimisation dynamics, convergence speed, and potential instabilities such as divergence or oscillatory behaviour[10, 11].

The validation loss is computed periodically every 500 training steps by evaluating the model on a fixed held-out validation set. Validation loss measures the model's ability to generalise to unseen protein sequences and serves as the primary indicator of upstream modelling performance[10, 28].

The validation loss is computed as:

$$\mathcal{L}_{\text{val}} = -\frac{1}{|D_{\text{val}}|} \sum_{(\tilde{\mathbf{x}},\mathbf{x})\in D_{\text{val}}} \sum_{i\in M} \log p_\theta(x_i \mid \tilde{\mathbf{x}}) \tag{3.5}$$

Algorithm 1 Cramming-Style Training for Protein Masked Language Modelling

Require: Training dataset $\mathcal{D}_{\text{train}}$, validation dataset $\mathcal{D}_{\text{val}}$
Require: Model f_θ initialized with parameters θ_0
Require: Maximum sequence length $L_{\max} = 512$
Require: Physical batch size $B_{\text{phys}} = 12$
Require: Gradient accumulation steps $G_{\text{accum}} = 24$
Require: Total training steps $T = 10{,}000$
Require: Peak learning rate $\eta_{\max} \in \{4 \times 10^{-4}, 1 \times 10^{-3}\}$
Require: Warm-up steps $W \in \{1000, 2000\}$
Require: Masking probability $p_{\text{mask}} = 0.25$
Require: Gradient clipping threshold $C = 0.5$

Initialize:
$\theta \leftarrow \theta_0$
Initialize AdamW optimiser[20] with $\beta_1 = 0.99$, $\beta_2 = 0.98$, $\epsilon = 10^{-12}$
accum_loss $\leftarrow 0$

for $t = 1$ **to** T **do**

    Phase 1: Gradient Accumulation
    for $g = 1$ **to** G_{accum} **do**
        Sample micro-batch $\mathcal{B} = \{\mathbf{x}^{(i)}\}_{i=1}^{B_{\text{phys}}}$ from $\mathcal{D}_{\text{train}}$
        Tokenize and truncate each $\mathbf{x}^{(i)}$ to length $L \leq L_{\max}$

        Apply masking:
        for each sequence $\mathbf{x}^{(i)}$ in $\mathcal{B}$ **do**
            Sample mask positions $M^{(i)} \subset \{1, \ldots, L\}$ with $|M^{(i)}| = \lfloor p_{\text{mask}} \cdot L \rfloor$
            Construct corrupted input $\tilde{\mathbf{x}}^{(i)}$ by replacing tokens at $M^{(i)}$ with `[MASK]`
        end for

        Forward pass: $\hat{\mathbf{y}} \leftarrow f_\theta(\tilde{\mathcal{B}})$
        Compute MLM loss:

$$\mathcal{L}_{\text{batch}} \leftarrow -\frac{1}{B_{\text{phys}}} \sum_{i=1}^{B_{\text{phys}}} \frac{1}{|M^{(i)}|} \sum_{j \in M^{(i)}} \log p_\theta(x_j^{(i)} \mid \tilde{\mathbf{x}}^{(i)})$$

        Scale loss: $\mathcal{L}_{\text{scaled}} \leftarrow \mathcal{L}_{\text{batch}} / G_{\text{accum}}$
        Backward pass: accumulate gradients using $\nabla_\theta \mathcal{L}_{\text{scaled}}$
        accum_loss $\leftarrow$ accum_loss $+ \mathcal{L}_{\text{batch}}$
    end for

    Phase 2: Parameter Update
    Clip gradients: $\mathbf{g} \leftarrow \text{clip}(\nabla_\theta, C)$

Algorithm 2 Cramming-Style Training (continued)

Compute learning rate:
if $t \leq W$ **then**
  $\eta \leftarrow \eta_{\max} \cdot (t/W)$
else
  $\eta \leftarrow \eta_{\max} \cdot \frac{T-t}{T-W}$
end if
Update parameters: $\theta \leftarrow \text{AdamW}(\theta, \mathbf{g}, \eta)$
Reset accumulated gradients

Phase 3: Logging, Validation, and Checkpointing
if $t \bmod 10 = 0$ **then**
  Log training loss: $\mathcal{L}_{\text{train}} \leftarrow \text{accum_loss}/G_{\text{accum}}$
  accum_loss $\leftarrow 0$
end if
if $t \bmod 500 = 0$ **then**
  Evaluate on $\mathcal{D}_{\text{val}}$ for 100 validation batches
  Compute $\mathcal{L}_{\text{val}}$ and $\text{PPL}_{\text{val}} = \exp(\mathcal{L}_{\text{val}})$
end if
if $t \in \{100, 500, 1000, 5000, 9000, 9500, 10000\}$ **then**
  Save checkpoint: $\text{save}(\theta, t)$
end if
end for

return Final model parameters θ_T

where D_{val} is the validation dataset and M is the set of masked positions in each sequence.

3.14.2 Validation Perplexity

Validation perplexity is the primary upstream evaluation metric[10, 19, 28] and is computed as the exponential of the validation loss:

$$\text{PPL}_{\text{val}} = \exp(\mathcal{L}_{\text{val}}) \tag{3.6}$$

Perplexity provides an interpretable measure of model uncertainty[4, 8]: a perplexity of k indicates that the model is, on average, as uncertain as if it were choosing uniformly among k equally likely amino acids at each masked position. Lower perplexity indicates better predictive performance and more effective modelling of protein sequence statistics.

The validation perplexity at the end of training step 10,000 is used as the primary metric for comparing architectures and depth configurations. All comparisons and conclusions are based on this upstream metric.

3.14.3 Training Efficiency Indicators

Training efficiency is assessed through the following indicators:

- Convergence Speed: The rate of validation loss decrease over the training budget[10, 11], measured by comparing validation loss at early steps of 1000–2000 and late steps of 8000–10,000 training stages.
- Training Time per Update: Wall-clock time per optimisation update, reflecting architectural computational efficiency[3, 7].
- Final Validation Performance: Validation perplexity at step 10,000[10], indicating the model's ultimate upstream performance within the fixed training budget.

3.14.4 Training Stability Indicators

Training stability is assessed qualitatively and quantitatively through the following indicators:

- Loss Trajectory Smoothness: Visual inspection of training and validation loss curves for oscillatory behaviour or sudden jumps[11].
- Gradient Norm Statistics: Monitoring of gradient norms to detect gradient explosion or vanishing gradient issues[25].
- Divergence Events: Detection of NaN or infinite loss values indicating training failure[10, 11].
- Sensitivity to Warm-up: Comparison of training behaviour across different warm-up schedules to assess optimisation robustness[11, 18].

These stability indicators are particularly important under cramming constraints[10, 11], where unstable training cannot be recovered through extended training runs or checkpoint rollback.

3.15 Downstream Evaluation Protocol

To complement the upstream masked language modelling analysis, this study evaluates the transferability of learned protein representations to downstream prediction tasks. Importantly, downstream evaluation does not introduce new model architectures or additional pre-training. Instead, it assesses the utility of representations learned by the pre-trained ESM2 and mLSTM models under cramming-style constraints[10, 11].

3.15.1 Motivation and Evaluation Scope

Upstream masked language modelling (MLM) performance provides insight into optimisation behaviour and sequence modelling capability under cramming-style constraints[10, 11]. However, MLM metrics alone do not directly measure whether learned representations are useful for practical protein prediction tasks[6]. To complement the upstream analysis presented earlier in this chapter, this section evaluates the transferability of representations learned by compact mLSTM[3] and ESM2[19] models to downstream protein property prediction tasks.

Downstream evaluation serves two complementary purposes in this thesis. First, it assesses whether models trained from scratch on UniRef50[2, 32] under strict computational budgets learn biologically meaningful representations beyond token-level likelihood optimisation[19, 28]. Second, it provides an additional comparison between recurrent and Transformer-based architectures that is independent of upstream loss and perplexity[6].

All downstream experiments are conducted using frozen representations extracted from the final upstream checkpoints[10, 28], ensuring that observed differences reflect representational quality rather than task-specific fine-tuning effects.

3.15.2 Frozen Representation Extraction

For downstream evaluation, the pre-trained upstream models are used as fixed feature extractors[10, 28]. All model parameters are frozen, and no additional fine-tuning of the language model weights is performed. Given an input protein sequence, the model produces a sequence of hidden representations, one per amino-acid token.

Following standard practice in protein language modelling[9, 28], a single fixed-size sequence embedding is obtained by mean pooling the final-layer token representations across the sequence length. This pooled embedding serves as a compact representation of the entire protein sequence and is used as input to downstream predictors.

This design ensures that downstream results reflect the quality of the learned representations rather than differences in downstream model capacity or training dynamics[6, 10].

3.15.3 Downstream Prediction Heads

For each downstream task, a lightweight regression head is trained on top of the frozen sequence embeddings. The prediction head consists of a single linear layer mapping the pooled embedding to a scalar output. This choice minimizes additional parameters and ensures that performance differences arise primarily from the upstream representations[6].

The regression head is trained using mean squared error (MSE) loss[6], with early stopping based on validation performance. Identical downstream training procedures and hyperparameters are applied across all upstream models to ensure fair comparison.

3.15.4 Downstream Tasks and Benchmarks

Downstream evaluation is conducted using protein property prediction tasks from the FLIP benchmark[6]. Specifically, the following datasets are used:

- AAV fitness prediction[6], which evaluates the ability to predict viral capsid fitness and is known to depend on long-range sequence interactions.
- GB1 mutational scanning[6], which measures binding-related fitness changes and emphasises local sequence effects.
- Meltome thermostability prediction[6], which assesses protein stability across temperature conditions.

These tasks span diverse biological properties and provide a complementary evaluation to upstream masked language modelling performance[6, 28].

3.16 Reproducibility and Transparency

To support reproducibility, the following measures are taken:

- Fixed Random Seeds: All experiments use fixed random seeds for dataset shuffling and parameter initialization.
- Version Control: All code, configuration files, and preprocessing scripts are maintained under version control.
- Checkpoint Preservation: Model checkpoints at step 10,000 are preserved for all experiments.
- Configuration Logging: Complete hyperparameter configurations are logged for each training run[10, 11].
- Hardware Specification: Detailed GPU specifications and driver versions are documented.

These measures enable independent verification of results and facilitate future extensions of this work.

3.17 Ethical Considerations

This research uses publicly available protein sequence data from the UniRef50 database[2, 32] and does not involve human subjects, personally identifiable information, or sensitive biological data. All experiments are conducted using institutional computational resources in accordance with university policies. No ethical concerns are identified for this methodological study.

3.18 Summary

This chapter presented the experimental methodology for investigating architectural depth–width trade-offs in mLSTM-based[3] protein language models trained under cramming-style computational constraints[10, 11]. The study consists of two complementary experimental components: a baseline comparison between 6-layer mLSTM[3] and 6-layer ESM2[19] models under matched parameter budgets, and a systematic depth scaling analysis of mLSTM variants of 1-layer, 6-layer and 12-layer with parameter parity[16].

The chapter detailed the dataset acquisition and preprocessing pipeline for UniRef50[2, 32], the masked language modelling training objective[8, 28] with 25% token masking, the complete experimental design with 16 total training runs, precise architectural specifications for all model variants, training configuration including batching strategy and AdamW optimisation settings[10, 20], upstream evaluation metrics focused exclusively on validation loss and perplexity[10, 19, 28], and a formal algorithmic description of the cramming-style training procedure[10, 11].

All experiments are conducted under strict cramming constraints[10, 11] using a single NVIDIA Tesla V100-PCIE-16GB GPU with a fixed training budget of 10,000 optimisation updates. Controlled variation of learning rate and warm-up duration[11, 18] enables assessment of optimisation sensitivity across architectures and depth configurations.

Critically, this study focuses exclusively on upstream masked language modelling performance. Downstream task evaluation on protein function prediction benchmarks from the FLIP benchmark[6] serves as a complementary analysis. All comparisons and conclusions are based on intrinsic metrics computed during pre-training: validation loss, validation perplexity, convergence speed, and training stability[10, 11].

The next chapter presents the experimental results and analysis derived from this methodology, including training dynamics, final upstream performance metrics, and architectural comparisons.

4 Results and Key Findings

This chapter reports the empirical findings of the study based on the experimental design presented in Chapter 3. The results are organised to directly address the research objectives, beginning with evidence of reliable training execution (stability, logging, checkpointing, and reproducibility), followed by quantitative upstream performance outcomes (validation loss, validation perplexity, and model comparisons).

4.1 Dataset Context and Protein Sequence Constraints

All reported results in this chapter are obtained from upstream masked language model training on protein sequences sampled from the UniRef50 dataset[2, 32], as described in Chapter 3. UniRef50 provides clustered protein sequences at 50% sequence identity, making it suitable for large-scale representation learning while reducing redundancy.

A key experimental constraint in this study is the handling of long protein sequences. Protein sequences naturally vary in length, and many UniRef50 sequences exceed the maximum context length that can be processed efficiently under limited GPU memory. Therefore, sequences were filtered and truncated to a maximum token length of $L_{\max} = 512$ tokens, ensuring consistent input dimensionality across models and training runs. In this setting, a "token" corresponds to an amino-acid symbol, including special tokens used for masked language modelling and the masked language model objective is evaluated on these tokenised protein sequences.

This sequence length constraint is particularly important under the cramming-style training budget, which consists of 10,000 optimisation steps on a Tesla V100-PCIE-16GB GPU, because it directly affects memory consumption, batch formation, and training speed. As a result, the training stability, validation trends, and final perplexity outcomes reported in the remainder of this chapter should be interpreted in the context of protein sequence modelling with bounded sequence length. Therefore, to ensure clear alignment with this objective, results in this chapter are reported in three stages:

1. Training execution evidence: confirmation that training ran reliably under the defined constraints and monitoring configuration.

2. Quantitative performance trends: validation loss and validation perplexity curves throughout training.

3. Comparative analysis: final performance comparisons between ESM2 and mLSTM variants, including checkpoint-based statistics and depth exploration.

This staged reporting ensures that later performance comparisons can be interpreted as meaningful architectural differences rather than artefacts of unstable optimisation or inconsistent experimental execution.

4.2 Training Monitoring and Execution Evidence

This section presents training-execution evidence derived from experiment logs and tracking outputs. These results confirm that the experimental pipeline executed as intended, enabling valid comparison of architectures under identical training constraints.

4.2.1 Logging and Experiment Tracking

Training runs were instrumented with structured console logs and Weights & Biases (W&B) tracking[5]. Before optimisation began, the training system consistently reported key configuration settings, including UniRef50 dataset loading, streaming data loader initialisation, tokenizer setup for protein sequences, maximum sequence length ($L_{\max} = 512$), optimiser and scheduler parameters, and runtime configuration.

During training, W&B tracked learning-rate progression, training loss, validation loss, update steps, and validation perplexity for the upstream masked language modelling objective. This monitoring setup enabled continuous inspection of learning dynamics while ensuring that the reported results remain traceable to the underlying protein sequence training runs.

4.2.2 Learning Rate Schedule Behaviour

Training used a linear warm-up followed by a linear decay schedule as specified in Chapter 3. Warm-up duration was varied across experiments ($W \in \{1000, 2000\}$) to test sensitivity to this hyperparameter. Log traces confirm that the learning rate increased smoothly during warm-up and then decayed gradually across the remaining optimisation steps, supporting stable early training dynamics under cramming constraints.

4.3 Validation Frequency and Monitoring

Validation was performed at regular intervals of 500 optimisation steps using a fixed validation budget of 100 validation batches sampled from the UniRef50 validation split. This ensured consistent evaluation across experiments while limiting evaluation overhead under the cramming budget.

Because UniRef50 protein sequences exhibit variable length distributions even after filtering to $L_{\max} = 512$, fixing the number of validation batches provides a stable and comparable estimate of validation loss and perplexity across architectures and hyperparameter settings.

4.3.1 Checkpointing and Run Completion

Model checkpoints were saved at steps 100, 500, 1000, 5000, 9000, and 10,000, with the final checkpoint at step 10,000 used for upstream evaluation. Training logs confirm that checkpoints were successfully written according to the predefined schedule and that runs completed the full $T = 10{,}000$ optimisation steps without interruption. This ensures that upstream comparisons are based on fully trained models under a consistent stopping criterion.

4.3.2 Training Stability Mechanisms and Observed Stability

Multiple stability mechanisms were active throughout training, including mixed precision training of FP16 computation with FP32 parameter updates[23], global gradient norm clipping (0.5)[25], and a Pre-LayerNorm configuration[18, 35].

These mechanisms are particularly relevant in the context of protein sequence modelling with long contexts of $L_{\max} = 512$, where memory pressure and gradient instability can become more pronounced under limited GPU resources. Across the reported training runs, optimisation remained stable, with no evidence of divergence, NaN losses, or failure to complete scheduled training steps. This stability is essential for interpreting later performance comparisons as meaningful architectural differences rather than training artifacts.

4.4 Validation Loss and Perplexity Results

This section reports upstream *protein* masked language modelling (MLM) performance on UniRef50 sequences using validation loss and validation perplexity, as defined in Chapter 3. In this thesis, the MLM objective measures how well a model reconstructs *masked amino-acid tokens* from bidirectional context in a protein sequence. A lower MLM loss and a lower perplexity indicates that the model assigns higher probability to the correct residue identity at masked positions, capturing statistical regularities of protein sequences such as conserved motifs, local residue patterns, and sequence-context dependencies present in UniRef50.

All experiments were trained under the same cramming-style budget of $T = 10{,}000$ optimisation steps, using a linear warm-up followed by linear decay learning-rate schedule. Validation was performed every 500 steps on 100 batches drawn from the UniRef50 validation split. All protein sequences were truncated to a maximum length of 512 tokens. Each token represents either an amino acid or a special symbol needed for the model. This sequence length was selected to work within GPU memory limits and to ensure fair comparison across all experiments.

Figure 4.1 summarises the learning dynamics of the two baseline models matched to

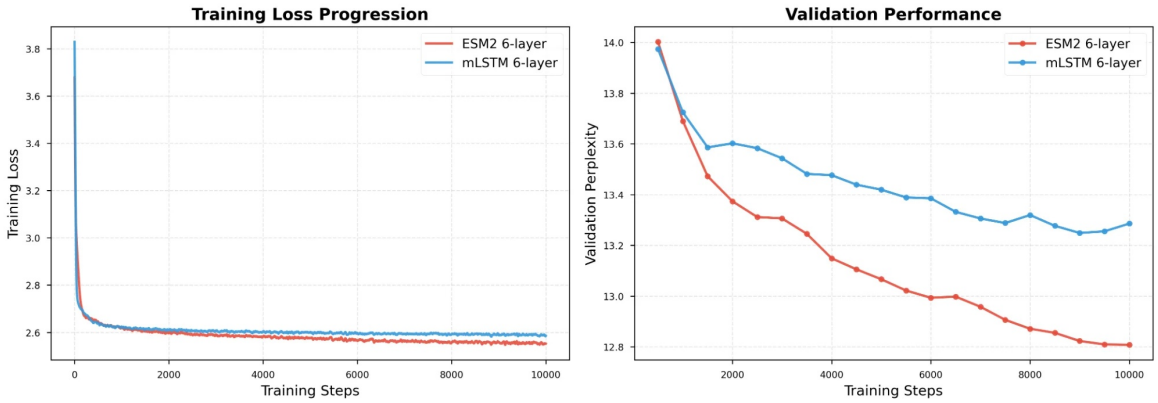


Figure 4.1: Training loss (left) and validation perplexity (right) across optimisation steps for the parameter-matched ESM2 (6-layer baseline) and mLSTM 6-layer, trained on UniRef50 protein sequences under the MLM objective.

the parameters of the ESM2[19] 6-layer baseline and mLSTM[3] 6-layer when trained from scratch on UniRef50 protein sequences. The left plot shows the evolution of the MLM training loss, computed on masked amino-acid positions, while the right plot reports validation perplexity, which provides an interpretable measure of uncertainty over the amino-acid vocabulary at masked sites. In this context, perplexity can be understood as the model's effective number of plausible amino choices at masked positions. However, lower perplexity implies sharper and more accurate residue predictions.

Across both architectures, the training loss decreases sharply during the early phase of optimisation and then transitions into a slower improvement regime. This behaviour is consistent with training under a warm-up and decay learning-rate schedule, where initial updates rapidly establish a useful representation space for protein tokens, followed by more incremental updates as the learning rate decays. Importantly, both curves remain smooth throughout the training horizon, suggesting stable optimisation despite the large-scale diversity of UniRef50 sequence length. This stability aligns with the design choices described in Chapter 3, including mixed-precision training[23], gradient clipping[25], and Pre-LayerNorm[18].

In the later portion of training, both models converge to a similar loss range, but the ESM2 baseline model trends slightly lower than the mLSTM model. Even when differences in loss are modest, they can correspond to consistent differences in token-level predictive quality across many masked positions, accumulated over a large validation set of protein sequences.

The validation perplexity curves show steady improvement for both architectures, indicating increasing predictive accuracy on held-out UniRef50 sequences. The largest gains occur during the first few thousand steps, after which both models continue to improve more gradually. This pattern suggests that early training captures broad regularities of protein sequences such as common residue dependencies and structural

patterns, while later training refines more specific dependencies.

A visible and persistent gap between the two perplexity curves is maintained across most checkpoints, indicating that the relative performance ordering is consistent throughout optimisation rather than being driven solely by late-stage training effects. At the same time, both models exhibit monotonic improvement and complete training without divergence, demonstrating that both the attention-based Transformer (ESM2) and recurrent (mLSTM) can learn meaningful protein sequence features from UniRef50 under the fixed cramming budget.

Overall, Figure 4.1 provides evidence that (i) upstream protein MLM training is stable under the specified constraints, (ii) both architectures extract learnable structure or features from UniRef50 sequences within 10,000 updates, and (iii) a consistent separation in validation perplexity motivates the more detailed comparisons in the following sections, including hyperparameter sensitivity and depth scaling of mLSTM variants.

4.5 Model Comparison Results: ESM2 vs. mLSTM

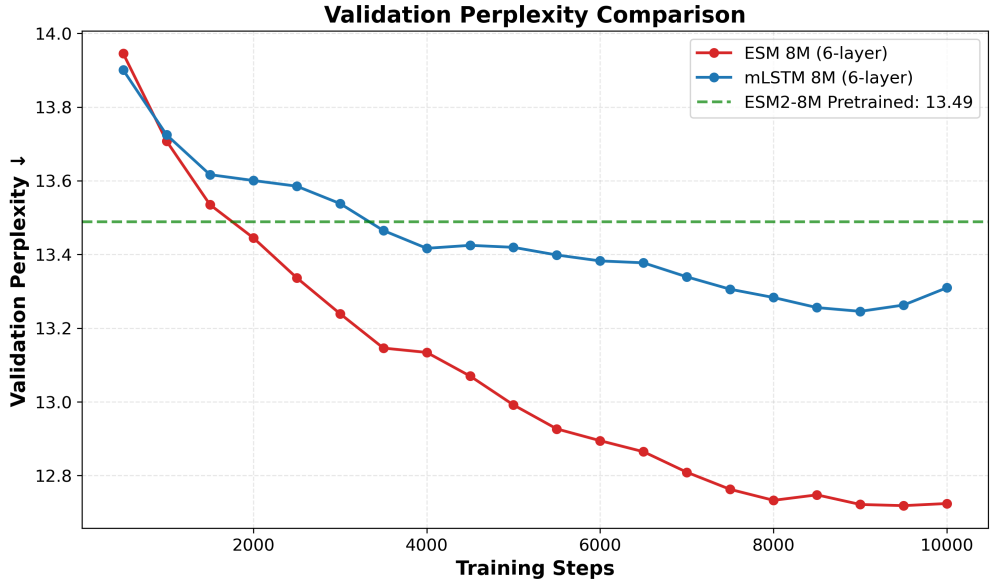


Figure 4.2: Validation perplexity comparison between ESM2-8M (6-layer baseline) and mLSTM-8M (6-layer) on UniRef50 protein sequences under the masked language modelling objective. The dashed line indicates the reported perplexity of the pre-trained ESM2-8M baseline.

Table 4.1 and Figure 4.2 provide a direct upstream comparison of the two parameter-matched baselines under identical training constraints. Both models were trained on UniRef50 protein sequences with the same maximum sequence length of $L_{\max} = 512$ and the same masking strategy, where models predict randomly hidden amino acids

Table 4.1: Final upstream validation results after 10,000 updates on UniRef50 protein MLM. Lower is better for both loss and perplexity.

Model	Learning Rate	Warm-up	Final Val. Perplexity
ESM2-8M (6-layer baseline)	4×10^{-4}	1000	12.8873
mLSTM-8M (6-layer)	4×10^{-4}	1000	13.3160

from the remaining sequence context. The reported perplexity measures prediction accuracy on validation sequences not seen during training.

Across the training horizon, both architectures show the characteristic pattern of rapid early gains followed by slower improvement, consistent with constrained training under the cramming regime. The key observation is that the ESM2 baseline maintains a lower validation perplexity than the mLSTM model throughout most checkpoints. At the final checkpoint ($t = 10{,}000$) for the representative configuration shown in Table 4.1, the ESM2 model reaches a validation perplexity of 12.8873, while the mLSTM model reaches 13.3160. This difference indicates that, with the same hyperparameters and training time, the ESM2 model predicts slightly higher amino acids than the mLSTM model on unseen protein sequences.

Crucially, this comparison is not intended to imply categorical superiority of one architecture family over another; rather, it provides an empirical reference point for the thesis objective. Since both models are trained under identical data, sequence length constraints, and optimisation schedules, the observed differences can be interpreted as differences in how efficiently each architecture learns amino acid patterns from UniRef50 within a limited number of training steps. The remainder of this chapter builds on this reference comparison by analysing mLSTM depth scaling under the same protein MLM setting.

4.6 Depth Exploration Results: mLSTM

Beyond the baseline comparison, additional experiments evaluated the effect of depth on mLSTM performance under the same cramming budget. Figure 4.3 summarises validation perplexity trends across mLSTM depth configurations, learning rates, and warm-up settings.

The depth exploration results indicate that mLSTM models can benefit from increased depth, with deeper configurations achieving lower validation perplexity than shallower variants in several hyperparameter settings. This behaviour is consistent with the general expectation that increasing depth can support richer hierarchical representation learning when optimisation remains stable.

Despite these depth-related improvements as shown in Table 4.3 and Figure 4.3, the best-performing mLSTM configurations in this study exhibit slightly higher validation

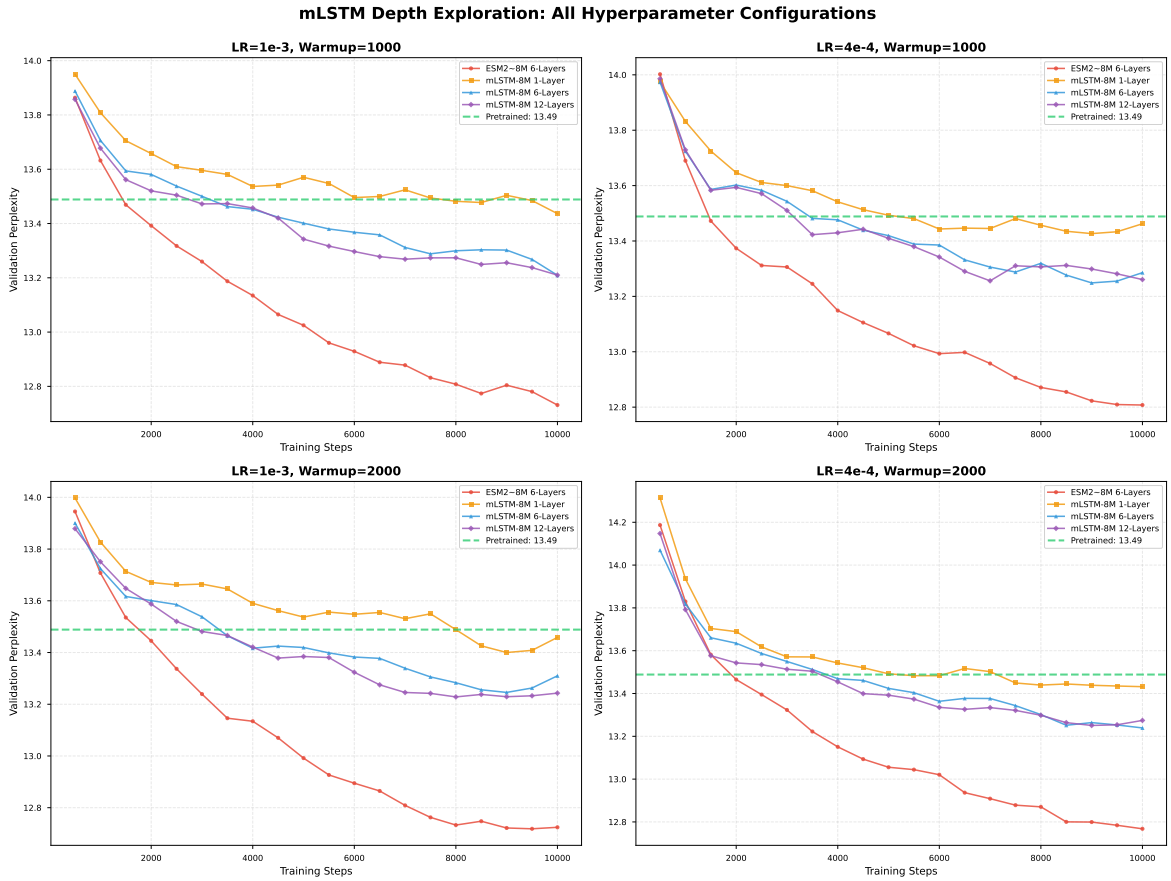


Figure 4.3: Validation perplexity trends for mLSTM depth exploration across learning rates and warm-up settings.

perplexity and validation loss compared to the ESM2 baseline under the same training budget. In the depth-matched comparison (both models approximately parameter-matched at ∼8M parameters and trained for $T = 10{,}000$ optimisation steps), the strongest 6-layer mLSTM setting achieved a final validation perplexity of 13.2948, while the best 6-layer ESM2 configuration achieved 12.8123. This corresponds to a validation perplexity difference of:

$$\Delta\text{PPL} = 13.2948 - 12.8123 = 0.4825.$$

A similar pattern is observed in validation loss, where the best 6-layer mLSTM result (2.5868) is above the ESM2 baseline (2.5496), giving:

$$\Delta\mathcal{L} = 2.5868 - 2.5496 = 0.0378.$$

Across the full depth sweep, the shallow mLSTM models (1-layer) show larger gaps relative to the Transformer baseline, suggesting that very low depth may limit representational hierarchy even when width is increased to maintain a fixed parameter budget. The deeper mLSTM variants (12-layer) reduce this gap in some settings and

can approach the performance of the balanced 6-layer models, although their results remain sensitive to the learning rate and warm-up configuration. Overall, these findings suggest that depth scaling is a meaningful factor for mLSTM performance under cramming constraints, while the Transformer baseline provides a strong reference point for comparison in the experimental setting considered in this thesis.

4.6.1 Hyperparameter Sensitivity Analysis for mLSTM

To further examine training behaviour under cramming-style constraints, we analyse the effect of learning rate and warm-up duration on a fixed 6-layer mLSTM architecture. Figure 4.4 presents the training loss (left) and validation perplexity (right) trajectories for four hyperparameter configurations, combining two learning rates (1×10^{-3} and 4×10^{-4}) with two warm-up durations (1000 and 2000 steps).

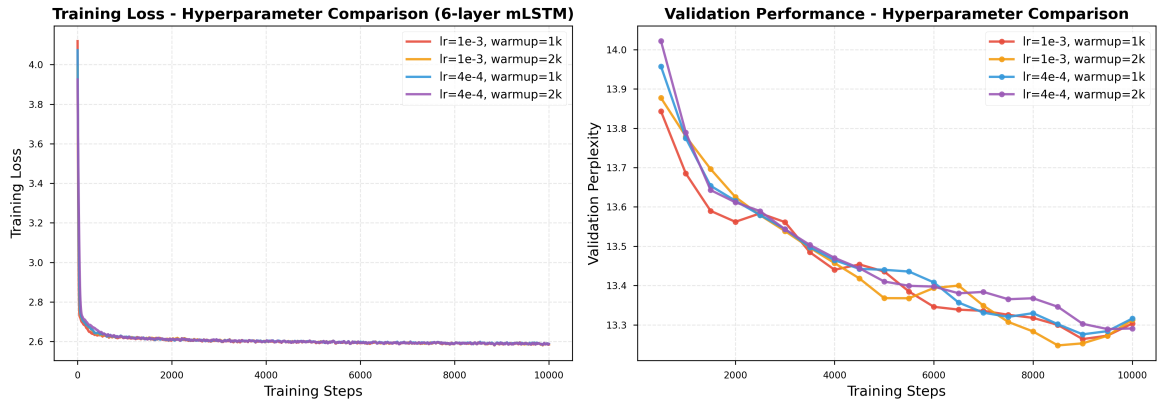


Figure 4.4: Training loss (left) and validation perplexity (right) for the 6-layer mLSTM under different learning rate and warm-up configurations.

Across all configurations, the training loss exhibits a rapid decrease during the initial optimisation phase, followed by a gradual convergence toward a stable range. This behaviour is consistent with expectations under a linear warm-up followed by learning-rate decay, and indicates that all tested hyperparameter settings allow the model to optimise effectively within the fixed training budget. Importantly, the loss curves remain smooth and closely aligned throughout training, suggesting that the optimisation process is robust to moderate variations in learning rate and warm-up duration for this architecture.

While training loss trajectories are nearly indistinguishable across configurations, the validation perplexity curves reveal more nuanced differences. In the early stages of training, configurations with shorter warm-up durations tend to reduce perplexity more rapidly, reflecting faster initial adaptation. However, as training progresses, these differences diminish, and all configurations converge toward a similar perplexity range by the final checkpoints.

Among the evaluated settings, the combination of a learning rate of 1×10^{-3} with a warm-up duration of 2000 steps achieves the lowest validation perplexity at the end of training. Nevertheless, the overall spread between configurations remains small, on the order of a few tenths of a perplexity point. This indicates that, for the 6-layer mLSTM under cramming constraints, performance is relatively insensitive to the specific choice of learning rate and warm-up duration within the explored range.

Taken together, these results suggest that the mLSTM architecture exhibits stable and predictable optimisation behaviour under constrained training budgets. Hyperparameter choices primarily influence early training dynamics rather than final convergence, supporting the use of standardized optimisation settings in subsequent architectural comparisons. This robustness is particularly relevant in the context of protein masked language modelling on large-scale sequence datasets, where extensive hyperparameter tuning may be impractical under strict computational limits.

4.7 Supplementary Upstream Results

4.7.1 Complete Checkpoint Performance

Table 4.2 presents the complete validation perplexity results for all training configurations across all logged checkpoints. These results provide detailed tracking of model performance throughout the training process for each hyperparameter combination explored in this study.

Table 4.2: Validation perplexity at logged checkpoints for all training configurations. Lower values indicate better upstream masked language modelling performance.

Arch.	**Depth**	**LR**	**Warmup**	**Step 1k**	**Step 5k**	**Step 10k**
ESM2 8M	6	1×10^{-3}	1000	13.6274	13.1066	**12.8123**
mLSTM 8M	1	1×10^{-3}	1000	13.7662	13.6050	13.4274
mLSTM 8M	1	1×10^{-3}	2000	13.7963	13.5591	13.4384
mLSTM 8M	1	4×10^{-4}	1000	13.8623	13.5282	13.4446
mLSTM 8M	1	4×10^{-4}	2000	13.8169	13.5472	13.4337
mLSTM 8M	6	1×10^{-3}	1000	13.7034	13.4593	13.3301
mLSTM 8M	6	1×10^{-3}	2000	13.7890	13.3752	13.3025
mLSTM 8M	6	4×10^{-4}	1000	13.7375	13.4270	13.3213
mLSTM 8M	6	4×10^{-4}	2000	13.7338	13.3924	13.3034
mLSTM 8M	12	1×10^{-3}	1000	13.6778	13.3640	13.1960
mLSTM 8M	12	1×10^{-3}	2000	13.7094	13.3760	13.2460
mLSTM 8M	12	4×10^{-4}	1000	13.6756	13.4050	13.2569
mLSTM 8M	12	4×10^{-4}	2000	13.7666	13.4132	13.2707

4.7.2 Final Upstream Performance Results

Table 4.3 presents the final validation performance metrics for all training configurations after 10,000 optimisation steps. These results summarize the upstream masked language modelling performance across all architectural variants and hyperparameter combinations explored in this study. The best-performing configuration for each architecture variant is highlighted in bold.

Table 4.3: Final upstream validation performance after 10,000 optimisation steps. Lower is better for both validation perplexity and validation loss. All models are approximately parameter-matched (∼8M parameters).

Architecture (Depth, Par.)	**LR**	**Warmup**	**Val. Perplexity**	**Val. Loss**
ESM2 (6-layer, ∼8M)	1×10^{-3}	1000	**12.8123**	**2.5496**
	1×10^{-3}	2000	12.8158	2.5507
	4×10^{-4}	1000	12.8873	2.5562
	4×10^{-4}	2000	12.8167	2.5507
mLSTM (1-layer, ∼8M)	1×10^{-3}	1000	**13.4359**	**2.5979**
	1×10^{-3}	2000	13.4955	2.6024
	4×10^{-4}	1000	13.4544	2.5993
	4×10^{-4}	2000	13.4693	2.6004
mLSTM (6-layer, ∼8M)	1×10^{-3}	1000	13.2948	2.5874
	1×10^{-3}	2000	**13.2874**	**2.5868**
	4×10^{-4}	1000	13.3160	2.5890
	4×10^{-4}	2000	13.3060	2.5882
mLSTM (12-layer, ∼8M)	1×10^{-3}	1000	13.2507	2.5841
	1×10^{-3}	2000	**13.1605**	**2.5772**
	4×10^{-4}	1000	13.2904	2.5870
	4×10^{-4}	2000	13.3040	2.5881

Table 4.4: Statistical summary of validation perplexity across all configurations at each checkpoint. Lower values indicate better performance.

Metric	**Step 1k**	**Step 5k**	**Step 10k**
Best Performance	13.6274	13.1066	12.8123
Worst Performance	13.8623	13.6050	13.4446
Performance Gap	0.2349	0.4984	0.6323
Mean (All Models)	13.7394	13.4258	13.2912
Std. Deviation	0.0641	0.1206	0.1590
Best Model	ESM2 6-layer (LR=1×10^{-3}, W=1k)		
Worst Model	mLSTM 1-layer (LR=4×10^{-4}, W=1k)		

Table 4.5: Performance comparison between ESM2 baseline and mLSTM variants at step 10,000. Gap represents the perplexity difference from ESM2 baseline.

Architecture	**Best Config**	**Perplexity**	**Gap from ESM2**	**Relative %**
ESM2 6-layer	LR=1×10^{-3}, W=1k	12.8123	–	–
mLSTM 12-layer	LR=1×10^{-3}, W=2k	13.1605	+0.3482	+2.72%
mLSTM 6-layer	LR=1×10^{-3}, W=2k	13.2874	+0.4751	+3.71%
mLSTM 1-layer	LR=1×10^{-3}, W=1k	13.4274	+0.6151	+4.80%
Best mLSTM	12-layer	13.1605	+0.3482	+2.72%
Worst mLSTM	1-layer	13.4446	+0.6323	+4.93%

Table 4.6: Validation perplexity progression for ESM2 and best/worst mLSTM configurations across training checkpoints.

Model	**Step 1k**	**Step 5k**	**Step 10k**	**Δ 1k→5k**	**Δ 5k→10k**	**Total Δ**
ESM2 6-layer	13.6274	13.1066	12.8123	-0.5208	-0.2943	-0.8151
mLSTM 12-layer (Best)	13.7094	13.3760	13.1605	-0.3334	-0.2155	-0.5489
mLSTM 6-layer	13.7890	13.3752	13.2874	-0.4138	-0.0878	-0.5016
mLSTM 1-layer (Worst)	13.7662	13.6050	13.4274	-0.1612	-0.1776	-0.3388

4.7.3 Comparative Analysis

The checkpoint statistics in Table 4.4 reveal several important patterns in model behaviour across the training trajectory:

Performance Gap Widening: The performance gap between the best (ESM2) and worst (mLSTM 1-layer) configurations increases substantially throughout training, growing from 0.23 perplexity points at step 1,000 to 0.63 at step 10,000. This 169% increase in the gap suggests that ESM2's architectural advantages compound over extended training.

Architecture Hierarchy: At the final checkpoint (Table 4.5), ESM2 achieves 12.81 perplexity, outperforming the best mLSTM variant (12-layer) by 2.72%. Within mLSTM architectures, depth correlates strongly with performance: the 12-layer variant achieves 13.16, while 6-layer and 1-layer variants reach 13.29 and 13.43 respectively, representing progressive degradation of 3.71% and 4.80% from ESM2 baseline.

Learning Dynamics: As shown in Table 4.6, ESM2 demonstrates superior learning efficiency, achieving a total perplexity reduction of 0.82 points compared to 0.55 for the best mLSTM (12-layer) and only 0.34 for the worst mLSTM (1-layer). Notably, ESM2's improvement rate remains relatively consistent across training phases (Δ1k→5k: -0.52,

Δ5k$\rightarrow$10k: -0.29), while mLSTM 6-layer shows diminished improvement in the later phase (Δ5k$\rightarrow$10k: -0.09), suggesting earlier convergence.

Variance Analysis: The increasing standard deviation across checkpoints (0.064 $\rightarrow$ 0.121 $\rightarrow$ 0.159) in Table 4.4 indicates growing performance divergence between configurations. This suggests that hyperparameter and architectural choices become more consequential as training progresses, with suboptimal configurations falling progressively further behind the best-performing model.

4.7.4 Summary of the Upstream Findings

This chapter reported upstream results for *protein* masked language modelling on UniRef50 under strict cramming-style constraints (single V100-16GB GPU and a fixed budget of $T = 10{,}000$ optimisation steps). All models were trained and evaluated on tokenised protein sequences bounded to a maximum sequence length of $L_{\max} = 512$, ensuring consistent computational conditions and comparable residue-level evaluation across architectures.

Across all configurations, training remained numerically stable. The combination of mixed precision training[23], gradient clipping[25], and Pre-LayerNorm[18] supported smooth loss trajectories without divergence, indicating that both attention-based ESM2-Transformer and recurrent mLSTM architectures can be trained from scratch on large-scale protein sequence data within a constrained update budget.

In terms of upstream MLM quality, both architectures achieved monotonic reductions in validation loss and validation perplexity on held-out UniRef50 sequences, demonstrating that each model learns informative sequence features for reconstructing masked residues. The ESM2 baseline consistently produced lower validation perplexity than the 6-layer mLSTM under matched parameter budgets, suggesting a modest advantage in protein sequence prediction accuracy within the explored optimisation settings. At the same time, the mLSTM models exhibited systematic improvements in several settings as depth increased from 1 to 12 layers under iso-parametric design, indicating that depth allocation can influence how effectively recurrent models utilise a fixed parameter budget for protein MLM.

Taken together, the results support the thesis objective of characterising Transformer and mLSTM behaviour under constrained protein MLM training. The ESM2 baseline provides a strong reference point under the UniRef50 and $L_{\max} = 512$ setting, while the depth exploration indicates that mLSTM performance is sensitive to architectural depth and optimisation configuration. The following sections build on these findings by examining depth-dependent trends more closely, including the role of warm-up duration and learning rate in shaping recurrent model convergence under the cramming regime.

4.8 Downstream Evaluation

To assess whether representations learned during upstream masked language modelling transfer effectively to downstream protein fitness prediction tasks, both architectures are evaluated on supervised benchmark datasets from the FLIP suite[6]. Given a protein sequence variant, the objective is to predict its experimentally measured fitness value whilst preserving the correct ranking of variants. Accurate rank preservation is essential for evaluating protein language models, as downstream fitness benchmarks measure how effectively learned representations capture functional relationships between sequence variants.

4.8.1 Downstream Tasks and Evaluation Protocol

Downstream performance is evaluated using three established protein fitness benchmarks from the FLIP suite[6]: GB1 mutational scanning, AAV capsid fitness prediction, and Meltome thermostability prediction. These tasks capture diverse biological properties and differ in the extent to which fitness depends on local versus long-range sequence interactions.

For each task, sequence representations are extracted from the pre-trained encoders and used as fixed inputs to supervised regression models. The encoder parameters remain frozen during downstream training, ensuring that performance reflects the intrinsic quality and transferability of the learned representations rather than task-specific fine-tuning effects.

Performance is evaluated using Spearman's rank correlation coefficient ρ and mean squared error (MSE). Spearman correlation measures monotonic agreement between predicted and true fitness values and serves as the primary metric because downstream protein fitness prediction tasks depend primarily on correct ranking of sequence variants rather than absolute prediction scale. Mean squared error provides complementary information about absolute prediction accuracy. Higher Spearman correlation and lower MSE indicate better performance.

4.8.2 Quantitative Results

Table 4.7 summarises downstream performance for parameter-matched ESM2 and mLSTM models trained under identical cramming constraints. ESM2 achieves higher Spearman correlation on all three tasks, with the largest advantage on AAV, a moderate advantage on GB1, and near parity on Meltome.

Both architectures successfully transfer representations learned during upstream training to downstream fitness prediction tasks. This confirms that cramming-style pre-training on UniRef50 produces representations with measurable biological utility.

Table 4.7: Downstream protein fitness prediction performance using frozen representations and linear regression heads. Higher Spearman correlation and lower MSE indicate better performance.

Task	Model	Spearman ρ	MSE
AAV	ESM2	0.59	6.35
	mLSTM	0.39	8.41
GB1	ESM2	0.57	1.68
	mLSTM	0.52	1.52
Meltome	ESM2	0.60	48.59
	mLSTM	0.58	47.91
Average	ESM2	**0.58**	**18.87**
	mLSTM	0.50	19.28

However, the magnitude of the performance gap varies substantially across tasks, indicating that architectural inductive biases interact differently with distinct sequence-function relationships.

4.8.3 Architectural Comparison Across Tasks

Figure 4.5 presents a direct comparison of Spearman correlation between ESM2 and mLSTM across all downstream tasks.

On AAV, shown in Figure 4.5a, ESM2 achieves substantially higher correlation of 0.59 compared to 0.39 for mLSTM. This large performance gap indicates that viral capsid fitness prediction strongly benefits from global context modelling.

On GB1, shown in Figure 4.5b, the performance difference decreases, with correlations of 0.57 for ESM2 and 0.52 for mLSTM. Both models demonstrate effective rank preservation, indicating that this task is less sensitive to architectural choice.

On Meltome, shown in Figure 4.5c, the two architectures achieve nearly identical performance, with correlations of 0.60 for ESM2 and 0.58 for mLSTM. This suggests that thermostability prediction depends primarily on features captured equally well by both architectures.

These results reveal a consistent pattern: the architectural advantage of Transformers increases with the importance of long-range sequence interactions.

4.8.4 Task-Specific Analysis

Prediction scatter plots in Figure 4.6 provide qualitative insight into prediction behaviour and explain the observed correlation differences.

AAV capsid fitness prediction. Figure 4.6a shows that ESM2 predictions follow a clear monotonic trend aligned with the ideal diagonal. In contrast, mLSTM predictions

exhibit greater dispersion and weaker alignment, particularly in intermediate fitness regions. This indicates reduced rank preservation and explains the lower Spearman correlation.

AAV capsid fitness depends strongly on global structural constraints and long-range epistatic interactions. Transformer self-attention enables direct modelling of such interactions, whereas recurrent architectures must propagate information sequentially, which can be less effective under constrained training regimes.

GB1 mutational effect prediction. Figure 4.6b shows that both architectures capture the overall variant ordering effectively. The prediction distributions appear similar, with ESM2 demonstrating slightly stronger alignment with the diagonal. This suggests that both architectures learn useful representations for mutation effect prediction in the GB1 binding landscape.

The GB1 task originates from deep mutational scanning experiments measuring enrichment associated with protein binding activity. These effects are largely driven by local structural determinants within the protein domain, reducing reliance on modelling long-range sequence interactions.

Meltome thermostability prediction. Figure 4.6c shows nearly identical prediction behaviour for both architectures. Both models produce predictions closely aligned with the diagonal, consistent with their similar Spearman correlations.

Thermostability prediction is largely influenced by local residue properties and stability-related sequence patterns, allowing both architectures to perform effectively.

4.8.5 Relationship Between Upstream and Downstream Performance

The downstream results demonstrate that upstream masked language modelling quality influences downstream fitness prediction performance, but the relationship is task dependent. Although ESM2 achieves lower upstream validation perplexity, the downstream performance advantage varies across tasks.

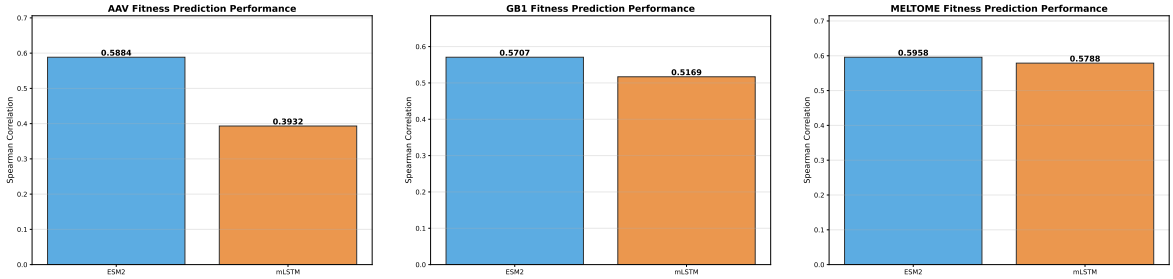
The largest downstream advantage occurs on AAV, where global context modelling is essential. In contrast, Meltome shows near parity, indicating that local feature representation is sufficient for accurate prediction.

This confirms that upstream perplexity alone does not fully determine downstream representation quality. Instead, architectural inductive biases and task characteristics jointly influence downstream transfer performance.

4.8.6 Architectural Implications

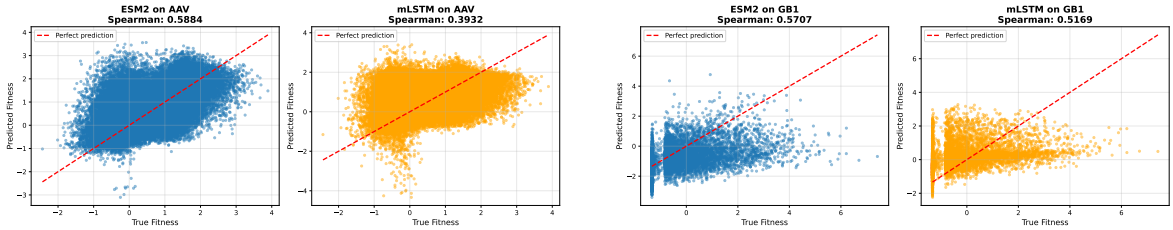
The downstream evaluation reveals three key architectural implications.

First, attention-based architectures provide clear advantages for tasks requiring modelling of long-range sequence interactions. This is demonstrated by the large per-



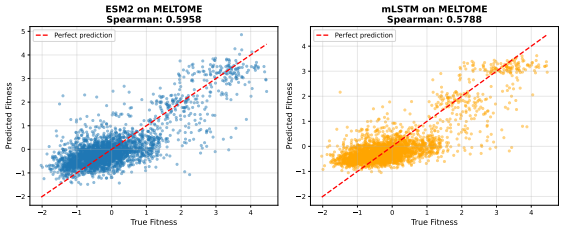
(a) AAV fitness prediction (b) GB1 fitness prediction (c) Meltome thermostability prediction

Figure 4.5: Spearman rank correlation comparison between ESM2 and mLSTM across downstream fitness prediction tasks.



(a) AAV fitness prediction

(b) GB1 fitness prediction



(c) Meltome thermostability prediction

Figure 4.6: Predicted versus true fitness for downstream tasks. Stronger alignment with the diagonal corresponds to higher rank correlation.

formance gap on AAV (Table 4.7).

Second, recurrent architectures remain competitive for tasks dominated by local sequence features, as shown by the near parity on Meltome (Table 4.7).

Third, cramming constraints amplify architectural efficiency differences. Under limited training time, architectures capable of rapidly learning global representations achieve stronger downstream performance.

4.8.7 Summary

Both ESM2 and mLSTM successfully learn transferable protein representations under cramming constraints. However, architectural differences produce task-dependent performance variations as shown in Table 4.7.

Transformer-based representations provide superior performance on tasks requiring global context modelling, whilst recurrent architectures remain competitive on tasks dominated by local features.

These findings complement the upstream analysis and demonstrate that architectural choice influences not only optimisation efficiency but also the functional utility of learned protein representations.

5 Discussion and Conclusion

5.1 Overview of Findings

This chapter synthesises the empirical results presented in Chapter 4 and situates them within the broader context of protein language modelling and efficiency-oriented training. The study investigated architectural depth–width trade-offs in modern recurrent protein language models under strict cramming-style constraints, using a fixed Transformer-based ESM2 configuration as a reference baseline.

Across all experiments, models were trained from scratch on protein sequences drawn from the UniRef50 dataset[2, 32] using a masked language modelling (MLM) objective[8]. Training was conducted under a fixed computational budget of 10,000 optimisation steps on a single GPU[10, 11], with all models constrained to approximately eight million parameters. This design ensured that observed differences in behaviour and performance could be attributed primarily to architectural and inductive-bias differences rather than scale or compute availability.

The results span both upstream masked language modelling performance and downstream protein property prediction tasks from the FLIP benchmark[6]. Together, these findings provide insight into how different architectures learn, optimise, and transfer representations under severe resource constraints.

5.2 Training Stability and Protein-Specific Considerations

A central prerequisite for meaningful architectural comparison is stable training. Across all reported experiments, both ESM2 and mLSTM models trained reliably, with no evidence of divergence, exploding gradients, or numerical instability. Stability was achieved through mixed precision training[23], global gradient norm clipping[25], and a Pre-LayerNorm configuration[18, 35], as described in Chapter 3.

Importantly, this stability was observed in the context of protein sequence modelling, which presents challenges distinct from natural language. Protein sequences in UniRef50 vary widely in length, encode long-range evolutionary dependencies, and use a specialised amino-acid vocabulary[19, 28]. The successful optimisation of both architectures under these conditions demonstrates that the cramming paradigm can be applied reliably to biologically realistic protein data, extending the findings of Frey et al.[10] to include modern recurrent architectures.

The smooth training curves and consistent validation behaviour observed across runs establish that subsequent differences in upstream and downstream performance reflect genuine architectural properties rather than artefacts of unstable optimisation.

5.3 Architectural Comparison Under Cramming Constraints

Upstream evaluation shows that both architectures exhibit similar learning dynamics, characterised by rapid early improvements followed by gradual refinement. This pattern is consistent with masked language modelling under linear warm-up and decay learning rate schedules, as reported by Geiping et al.[11] and Frey et al.[10] for Transformer-based cramming.

As shown in Table 4.1 and Figure 4.1, the Transformer-based ESM2 baseline achieves lower validation loss and perplexity at convergence across the explored hyperparameter settings. At matched depth and parameter count (six layers, ∼8M parameters), the best ESM2 configuration reaches a final validation perplexity of 12.81, compared to 13.29 for the best 6-layer mLSTM, a gap of 0.48 perplexity points (Table 4.5).

Rather than indicating categorical superiority, this result highlights how different inductive biases interact with constrained optimisation budgets. Self-attention provides direct access to global sequence context[34], which may be advantageous for modelling long-range dependencies under limited training time. In contrast, mLSTM relies on sequential matrix memory updates[3], which may require additional optimisation steps to reach comparable upstream performance. This interpretation is consistent with the broader observation that architectural efficiency matters more under cramming constraints than under unconstrained training[10, 11].

5.4 Depth–Width Trade-Offs in mLSTM Models

A primary contribution of this thesis is the systematic depth exploration of mLSTM architectures under a fixed parameter budget. As reported in Table 4.3 and Table 4.5, depth scaling in mLSTM consistently improves upstream performance: the 12-layer variant achieves a best perplexity of 13.16, compared to 13.29 for the 6-layer and 13.44 for the 1-layer variant.

Shallow mLSTM models converge reliably but exhibit higher validation perplexity, suggesting limited hierarchical capacity despite increased width. This finding is consistent with the expectation that depth enables hierarchical feature composition that cannot be replicated by simply widening layers[16]. Balanced configurations (six layers) provide a favourable trade-off between depth and optimisation efficiency, consistently outperforming shallow variants across hyperparameter settings (Table 4.2).

Deeper models (twelve layers) further reduce validation perplexity in several settings, indicating that increased depth can support richer hierarchical representations when optimisation remains stable. However, as shown in Table 4.6, the 12-layer mLSTM achieves a total perplexity reduction of only 0.55 points across training, compared

to 0.82 for ESM2, suggesting that deeper recurrent stacks may not utilise the full training budget as effectively. This is consistent with the known sensitivity of deeper recurrent architectures to learning rate and warm-up choices[3, 25], which is also reflected in the greater variance of 12-layer results across hyperparameter configurations (Table 4.4).

These findings demonstrate that depth scaling in recurrent protein language models is beneficial but non-trivial under cramming constraints, and extend prior depth scaling analysis beyond Transformer-centric architectures[10, 16].

5.5 Downstream Transfer and Functional Representation Quality

Beyond upstream MLM metrics, downstream evaluation provides insight into whether representations learned under cramming constraints capture biologically meaningful features. As reported in Table 4.7 and Figure 4.5, both ESM2 and mLSTM models demonstrate the ability to transfer pre-trained representations to supervised regression settings across three FLIP benchmark tasks[6].

Downstream performance varies substantially by task. On GB1 and Meltome, the performance gap between architectures is modest (0.05 and 0.02 Spearman correlation respectively, Table 4.7), indicating that both models capture local and stability-related sequence features effectively. On AAV fitness prediction, which depends more strongly on long-range epistatic interactions[6], ESM2 achieves substantially higher rank correlation (0.59 vs. 0.39), as shown in Figure 4.5a and Figure 4.6a.

This pattern is consistent with the known advantage of self-attention for modelling global sequence dependencies[19, 28, 34]. Recurrent architectures, which propagate context sequentially through time[3, 14], may capture such interactions less completely under a constrained training budget. The near-parity on Meltome (Figure 4.5c) confirms that this limitation is task-specific rather than general, and that mLSTM representations retain practical utility for tasks dominated by local sequence features.

5.6 Relationship Between Upstream and Downstream Performance

Comparing upstream and downstream results reveals that lower MLM perplexity does not guarantee proportional gains in downstream task performance. While ESM2 achieves a perplexity advantage of 0.48 points over the best 6-layer mLSTM (Table 4.5), the downstream performance gap on Meltome is only 0.02 Spearman correlation (Table 4.7), whereas on AAV it reaches 0.20. This task-dependence of the upstream-to-downstream relationship has been noted in prior work on protein language model

evaluation[6, 10], and the present results extend this observation to the comparison of recurrent and attention-based architectures under constrained training.

This finding reinforces the importance of downstream evaluation as a complementary assessment beyond intrinsic MLM metrics[6, 28], particularly when models are trained under constrained regimes where upstream performance differences may be numerically small but architecturally meaningful.

5.7 Implications for Efficient Protein Language Modelling

Taken together, the results demonstrate that modern recurrent architectures can be trained effectively on large-scale protein datasets under extreme computational constraints, extending the cramming paradigm[10, 11] beyond Transformer-centric approaches. Depth scaling in mLSTM models yields measurable upstream benefits (Table 4.3) and the architecture remains competitive on downstream tasks requiring local feature capture (Table 4.7).

At the same time, the Transformer-based ESM2 baseline provides a strong reference point across both upstream and downstream evaluations. These findings suggest that architecture choice should be guided by both computational constraints and target application characteristics. For tasks requiring long-range dependency modelling, the attention-based inductive bias of ESM2 provides a measurable advantage under cramming constraints[19, 34]. For tasks dominated by local sequence features, mLSTM represents a viable and computationally efficient alternative[3].

5.8 Limitations

Several limitations should be acknowledged. Each configuration was trained only once, limiting statistical characterisation across random seeds. This design choice is consistent with the cramming paradigm[10, 11] but means that reported perplexity values carry uncertainty that cannot be quantified without repeated runs. Downstream evaluation was restricted to three regression tasks from the FLIP benchmark[6], and no task-specific fine-tuning was performed, which may underestimate the practical utility of the learned representations. Additionally, the study considers a single dataset (UniRef50)[2, 32] and a fixed sequence length of 512 tokens, which may not capture the full diversity of protein sequence modelling challenges.

5.9 Future Work

Future work could extend downstream evaluation to additional structural and functional tasks[6], explore recurrence-specific optimisation strategies tailored to the mLSTM architecture[3], or examine longer training horizons to assess whether the up-

stream perplexity gap narrows with additional compute[15, 16]. Scaling experiments beyond the eight-million-parameter regime would further clarify how depth–width trade-offs evolve with increased model capacity, building on the scaling law framework established for Transformers[15, 16].

5.10 Key Contributions of the Thesis

This thesis makes the following contributions to the study of efficient protein modelling under resource-constrained budgets:

1. It presents a controlled empirical comparison between Transformer-based (ESM2)[19] and modern recurrent (mLSTM)[3] protein language models trained entirely from scratch under strict cramming-style constraints[10, 11], providing the first direct architectural comparison of this kind under a fixed 24-hour single-GPU budget.

2. It provides a systematic investigation of depth–width trade-offs in mLSTM architectures under a fixed parameter budget (Table 4.3, Table 4.5), offering empirical guidance for recurrent model design in protein sequence modelling that extends prior scaling analysis beyond Transformers[10, 16].

3. It demonstrates that stable and effective protein language model training is feasible under extreme resource constraints using large-scale biological data (UniRef50)[2, 32], without reliance on large pre-trained checkpoints, confirming and extending the findings of Frey et al.[10] to recurrent architectures.

4. It extends evaluation beyond upstream masked language modelling by incorporating downstream protein property prediction tasks from the FLIP benchmark[6] (Table 4.7, Figure 4.5), demonstrating task-dependent differences in representation transfer between architectural families.

5. It shows that upstream metrics such as validation perplexity are not sufficient predictors of downstream performance[6, 10], reinforcing the need for task-level evaluation when assessing efficient protein models trained under constrained regimes.

5.11 Conclusion

This thesis presented a systematic study of architectural depth scaling in mLSTM-based protein language models trained under cramming-style constraints[10, 11]. By combining upstream masked language modelling analysis with downstream functional evaluation on the FLIP benchmark[6], the study provides a comprehensive view of how architectural choices influence both optimisation behaviour and representation transfer under severe resource limitations.

The results reported in Table 4.3 and Table 4.7 show that modern recurrent architectures can learn transferable protein representations under extreme computational constraints, while depth scaling plays a critical role in achieving competitive upstream performance. The Transformer-based ESM2 baseline maintains a consistent advantage in both upstream perplexity and on downstream tasks requiring long-range dependency modelling, as demonstrated most clearly by the AAV results in Figure 4.5a and Figure 4.6a. However, the performance gap narrows substantially on tasks dominated by local sequence features (Figure 4.5c), confirming that the choice between recurrent and attention-based architectures should be informed by the requirements of the target application.

Overall, this work extends efficiency-oriented protein modelling beyond Transformer-centric approaches[10, 11, 19] and provides practical guidance for designing compact models under realistic computational budgets, contributing to more accessible and reproducible practices in computational protein modelling.

Bibliography

[1] Alley, E. C., Khimulya, G., Biswas, S., AlQuraishi, M., & Church, G. M. (2019). Unified rational protein engineering with sequence-based deep representation learning. *Nature Methods*, 16, 1315–1322.

[2] Bateman, A., Martin, M.-J., Orchard, S., Magrane, M., et al. (2021). UniProt: The universal protein knowledgebase in 2021. *Nucleic Acids Research*, 49(D1), D480–D489.

[3] Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J., & Hochreiter, S. (2024). xLSTM: Extended Long Short-Term Memory. arXiv:2405.04517.

[4] Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A neural probabilistic language model. *Journal of Machine Learning Research*, 3, 1137–1155.

[5] Biewald, L. (2020). Experiment tracking with Weights and Biases. *Software available from wandb.com*. `https://www.wandb.com`

[6] Dallago, C., Mou, J., Johnston, K.E., Wittmann, B.J., Bhattacharya, N., Goldman, S., Madani, A., and Yang, K.K. (2021). FLIP: Benchmark tasks in fitness landscape inference for proteins. *NeurIPS 2021 Datasets and Benchmarks Track*.

[7] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems*, 35.

[8] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*.

[9] Elnaggar, A., Heinzinger, M., Dallago, C., Rehawi, G., Wang, Y., Jones, L., Gibbs, T., Feher, T., Angerer, C., Steinegger, M., Bhowmik, D., & Rost, B. (2021). ProtTrans: Toward understanding the language of life through self-supervised learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10), 7112–7127. `https://doi.org/10.1109/TPAMI.2021.3095381`

[10] Frey, N. C., Joren, T., Ismail, A. A., Goodman, A., Bonneau, R., Cho, K., & Gligorijević, V. (2024). Cramming protein language model training in 24 GPU hours. *bioRxiv*. `https://doi.org/10.1101/2024.05.14.594108`.

[11] Geiping, J., & Goldstein, T. (2023). Cramming: training a language model on a single GPU in one day. In *Proceedings of the International Conference on Machine Learning*. PMLR.

[12] Gu, A., & Dao, T. (2023). Mamba: linear-time sequence modelling with selective state spaces. arXiv:2312.00752.

[13] Hochreiter, S., Beck, M., Pöck, T., Klambauer, G., & Kühn, L. (2024). The xLSTM architecture: Extended long short-term memory for sequence modelling. *arXiv preprint arXiv:2405.04517*.

[14] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

[15] Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., et al. (2022). Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*.

[16] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling Laws for Neural Language Models," arXiv preprint arXiv:2001.08361, 2020. :contentReference[oaicite:0]index=0

[17] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.

[18] Liu, X., Cheng, H., He, P., Chen, W., Wang, Y., Poon, H., & Gao, J. (2020). On the variance of the adaptive learning rate and beyond. *International Conference on Learning Representations*.

[19] Lin, Z., Akin, H., Rao, R., et al. (2023). Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637), 1123–1130.

[20] Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *Proceedings of the 7th International Conference on Learning Representations (ICLR)*.

[21] Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., et al. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873), 583–589.

[22] Meier, J., Rao, R., Verkuil, R., Liu, J., Sercu, T., & Rives, A. (2021). Language models enable zero-shot prediction of the effects of mutations on protein function. In *Advances in Neural Information Processing Systems*.

[23] Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., et al. (2018). Mixed precision training. *Proceedings of the 6th International Conference on Learning Representations (ICLR)*.

[24] Peng, B., Alcaide, E., Anthony, Q., Albalak, A., Arcadinho, S., Cao, H., et al. (2023). RWKV: Reinventing RNNs for the transformer era. *arXiv preprint arXiv:2305.13048*

[25] Pascanu, R., Mikolov, T., & Bengio, Y. (2013). On the difficulty of training recurrent neural networks. *Proceedings of the 30th International Conference on Machine Learning*, 1310–1318.

[26] Poli, M., Massaroli, S., Nguyen, E., Fu, D. Y., Dao, T., Baccus, S., Ermon, S., & Ré, C. (2023). Hyena Hierarchy: Towards Larger Convolutional Language Models. In *Proceedings of the International Conference on Machine Learning (ICML)*. PMLR.

[27] Rao, R., Bhattacharya, N., Thomas, N., Duan, Y., Chen, X., Canny, J., Abbeel, P., & Song, Y. S. (2019). Evaluating protein transfer learning with TAPE. In *Advances in Neural Information Processing Systems*.

[28] Rives, A., Meier, J., Sercu, T., Goyal, S., Lin, Z., Liu, J., et al. (2021). Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the National Academy of Sciences*, 118(15), e2016239118.

[29] Steinegger, M., & Söding, J. (2018). Clustering huge protein sequence sets in linear time. *Nature Communications*, 9, 2542.

[30] Steinegger, M., & Söding, J. (2019). Clustering huge protein sequence sets in linear time. *Nature Communications*, 9(1), 2542.

[31] E. Strubell, A. Ganesh, and A. McCallum, "Energy and Policy Considerations for Deep Learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 36453650, 2019. :contentReference[oaicite:2]index=2

[32] Suzek, B. E., Wang, Y., Huang, H., McGarvey, P. B., Wu, C. H., & the UniProt Consortium. (2015). UniRef clusters: A comprehensive and scalable alternative for improving sequence similarity searches. *Bioinformatics*, 31(6), 926–932.

[33] Tay, Y., Dehghani, M., Abnar, S., Shen, Y., Bahri, D., Pham, P., Rao, J., Yang, L., Ruder, S., & Metzler, D. (2022). Efficient transformers: A survey. *ACM Computing Surveys*, 55(6), 1–28.

[34] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*.

[35] R. Xiong, Y. Yang, D. He, K. Zheng, and S. Zheng, "On Layer normalisation in the Transformer Architecture," arXiv preprint arXiv:2006.04768, 2020.

[36] Yang, K. K., Wu, Z., & Arnold, F. H. (2021). Machine-learning-guided directed evolution for protein engineering. *Nature Methods*, 18, 687–694.

[37] Sun, Y., Dong, L., Huang, S., Ma, S., Xia, Y., Xue, J., et al. (2023). Retentive network: A successor to Transformer for large language models. *arXiv preprint arXiv:2307.08621*.